# Optimizing Dynamic-Shape Operator Libraries via Automatic Model-Specific Specialization on AI Accelerators

Deep-learning (DL) systems rely on expert-written operator libraries for high performance across dynamic input shapes. These libraries use shape-dependent schedule parameters, such as tile sizes and on-chip buffer layouts. Schedule-generic expert-written operator libraries preserve shape coverage with a single kernel but limit compile-time optimization, whereas schedule-fixed expert-written operator libraries precompile combinations of schedule parameters and incur binary bloat. Kernel-generation compilers do not resolve this trade-off because they generate new kernels rather than specialize existing libraries.

Although many schedule parameters are fixed for a given model, the corresponding model-specific constants recorded in Graph IR are absent from the LLVM IR of separately compiled C++ operator libraries. Runtime argument transmission through Python/C++ FFI does not make these values available as compile-time facts. Propagating model-specific constants from the framework enables specialization, but existing toolchains face two challenges: the Python/C++ semantic gap prevents constant information from reaching the native compiler, and shape checkers for dynamic dimensions introduce many early-exit paths that weaken constant propagation.

We present TilingInfer, a static-analysis framework that uses LLVM to propagate Graph IR shape information through existing C++ operator libraries and derive constant schedule parameters for kernel specialization and redundant-code elimination. To address the two challenges above, TilingInfer introduces two core designs. First, type-specific context-update rules construct C++ entry states directly from Graph IR shapes and attributes, avoiding analysis of Python/C++ FFI glue code. Second, a Conditional Return-Value Flow Graph identifies early-exit paths and prevents their states from affecting normal paths, preserving precision without analyzing every path.

On an Ascend NPU, Tiling achieves an average 1.06× operator speedup across diverse operators and an average 1.05× speedup across six representative inference scenarios. It provides larger benefits in the LLM decode phase and recommendation models, averaging a 1.10× speedup. On an NVIDIA GPU, it reduces a 542 MB binary size by 88.8–95.5% across 17 model variants.

CCS Concepts: • **Software and its engineering** → **Dynamic compilers**; • **Computer systems organization** → *Single instruction, multiple data*; • **Computing methodologies** → *Neural networks*; • **General and reference** → *Performance*.

Additional Key Words and Phrases: compilers, static analysis, deep learning, kernel specialization, constant propagation, AI accelerators, operator libraries

**ACM Reference Format:**

. 2026. Optimizing Dynamic-Shape Operator Libraries via Automatic Model-Specific Specialization on AI Accelerators. 1, 1 (July 2026), 25 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Deep-learning (DL) frameworks and serving systems [1, 3, 16, 41] rely heavily on expert-written operator libraries [4, 7, 8, 13, 20, 37]. These libraries must cover input dynamics from both cross-model heterogeneity (such as different algorithmic variants [2, 5, 26, 33] and architectural attributes [9, 32, 36]) and runtime variation in dimensions (such as

Author's Contact Information:

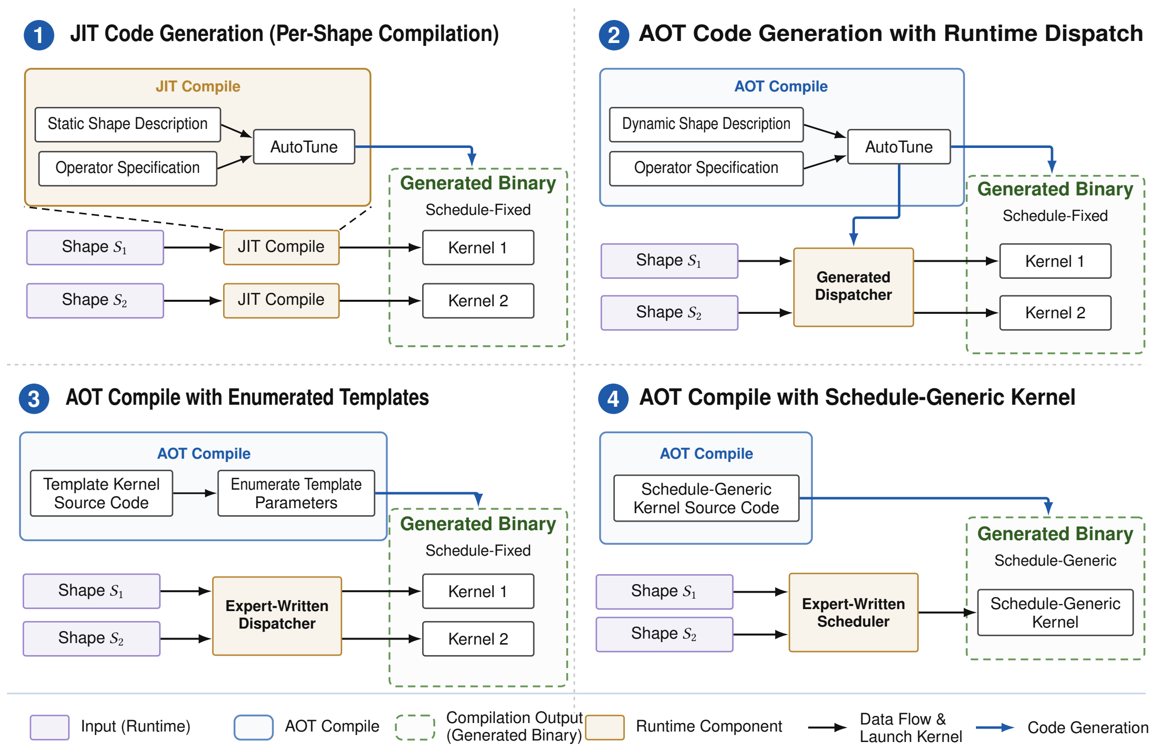



Fig. 1. Four ways to compile and execute dynamic-shape operators.

batch size and sequence length). The performance of each resulting operator configuration depends not only on its algorithm but also on its schedule parameters, including tile sizes, loop bounds, pipeline stages, and memory layout choices. Different shapes can have different optimal schedules. Consequently, when input shapes are dynamic, the operator implementation must select a high-performance schedule from the runtime shape.

Existing approaches can be classified by how their generated binaries represent schedule parameters. A schedule-fixed kernel compiles the key schedule parameters as constants, so each compiled kernel realizes a fixed schedule. A schedule-generic kernel retains the schedule parameters as runtime variables, allowing one compiled kernel to realize different schedules for different shapes. The four approaches in Figure 1 comprise three schedule-fixed approaches (①–③) and one schedule-generic approach (④).

AI compilers produce schedule-fixed kernels through schedule search and code generation. ① A static-shape AI compiler takes a concrete shape as input, searches for or autotunes a schedule for that shape, and generates a shape-specific kernel, often through a just-in-time (JIT) compilation path [29–31, 40]. An unseen shape triggers another round of schedule search and code generation. ② A dynamic-shape AI compiler instead searches for schedules over a declared shape range ahead of time. It generates a set of optimized, schedule-fixed kernels together with a dispatcher that maps each runtime shape in the range to an appropriate kernel [17, 27, 39, 42].

Expert-written operator libraries implement the other two approaches. ③ A schedule-fixed expert-written operator library encodes schedule parameters as C++ template parameters and enumerates their candidate values during ahead-of-time (AOT) compilation [7, 8, 37]. This process produces a set of schedule-fixed kernels, from which an expert-written dispatcher selects a template instance for each runtime shape. ④ A schedule-generic expert-written operator library instead AOT-compiles a schedule-generic kernel. At runtime, an expert-written host-side scheduler applies heuristic rules to the input shape, computes the schedule parameters, and passes them to the device kernel as runtime arguments.

AI compilers such as Triton and TileLang complement expert-written operator libraries but operate on a different compilation path [30, 31]. These code-generation workflows integrate specialization into the multi-level lowering of a high-level operator program. During lowering, the shape and schedule values selected for specialization are materialized and propagated as constants in the intermediate representations. This process generates a new specialized kernel implementation but does not directly specialize the existing C++/LLVM implementation of an expert-written operator library.

The two expert-written library designs expose a common specialization trade-off. Schedule-fixed kernels expose schedule parameters as compile-time constants, enabling optimizations such as constant folding, loop unrolling, and instruction scheduling; however, enumerating combinations of schedule parameters causes combinatorial code-size growth. Schedule-generic kernels use a compact binary to cover a large shape space, but representing schedule parameters as runtime variables limits the same compiler optimizations and can reduce performance. This trade-off becomes more pronounced as operator implementations grow more complex [8, 37] and accelerator architectures expose increasingly diverse programmable units [14, 18]. Both trends increase the number of schedule parameters and the size of their configuration space, further exacerbating the trade-off.

Our key observation is that most schedule parameters are runtime invariants once a model is deployed. Most deployment scenarios exhibit partial dynamics: only a limited set of dimensions, such as batch size and sequence length, vary at runtime, while model-specific attributes and dimensions, such as hidden size, head count, and head dimension, remain fixed. We refer to these fixed facts as model-specific constants. Current DL frameworks can recover model-specific constants through graph capture and symbolic shape inference [3, 17, 27, 42]. Mechanistically, model-specific constants determine the core tiling and pipeline structure constrained by on-chip resources and compute units, whereas dynamic dimensions primarily affect the number of executions of these tiles, their parallel partitioning, and tail handling. Consistent with this mechanism, our analysis in §2 finds that, across the studied operators, about 90% of schedule parameters are runtime invariants determined by model-specific constants and do not depend on the concrete runtime values of dynamic dimensions.

This observation suggests a novel way to address the specialization trade-off: using model-specific constants to specialize existing operator libraries. Inspired by this insight, we present **TilingInfer**, a static-analysis framework for LLVM-based operator libraries. It uses LLVM IR as a common representation for C++ libraries. TilingInfer first uses graph-level model constants to **infer** invariant **tiling** data (i.e., schedule parameters) for both schedule-fixed and schedule-generic implementations. For each Graph IR node(i.e., an operator invocation), TilingInfer extracts its arguments, shapes, and attributes to construct the entry state of the corresponding host-side LLVM IR function. This construction covers both concrete and symbolic dimensions. TilingInfer then propagates this constructed state information through the host-side LLVM IR programs to device-kernel launch sites. Finally, for a schedule-generic kernel, TilingInfer employ the inferred launch-site constant schedule parameters to specialize the kernel. For the schedule-fixed implementation, TilingInfer identifies template instances that cannot be reached and removes these instances.

Realizing TilingInfer faces two key challenges. **First**, reconstructing the calling context requires bridging **a cross-language semantic gap** between graph-level constants and symbolic shape information to the field-sensitive entry state of the corresponding C++ operator. Graph IR records constants and symbolic dimensions as typed shapes and attributes, whereas the separately compiled C++/LLVM code exposes nested object layouts and pointer/field accesses.

Bridging this gap through end-to-end cross-language analysis would require traversing framework dispatch, type conversion, object construction, and other Python/FFI machinery, making the analysis costly and framework-specific. TilingInfer instead uses type-specific context-update rules to precisely construct the required C++ entry state directly from Graph IR, avoiding analysis of Python/C++ FFI glue code. **Second**, operator host code contains many **shape checkers, leading to numerous early-exit paths**. Their early-exit paths just handle invalid inputs, report errors and skip the normal computation of schedule parameters. As a result, merging both states on early-exit paths and normal paths can pollute the normal state and reduce the precision of constant propagation. Meanwhile, analyzing every possible path separately would be too expensive for a large operator library. TilingInfer therefore uses a Conditional Return-Value Flow Graph (CRVFG) to identify early-exit paths before propagation based on . It excludes states from these paths when states from normal paths are merged. This preserves constants without analyzing every path separately.

This paper makes four contributions:

(1) We devise a mechanism that propagates graph-level shape information into the C++ compiler pipeline without analyzing FFI glue code. We formalize context-update rules that map Graph IR parameter types to the memory states of their underlying C++ implementations, allowing the mechanism to cover different PyTorch backends.
(2) We propose CRVFG-based early-exit path identification for precise constant-propagation analysis of operator host code while avoiding expensive path-sensitive analysis.
(3) We implement TilingInfer, a prototype system for specializing expert-written operator libraries on GPU and NPU platforms. It applies automatic partial evaluation to schedule-generic kernels to reduce scalar overhead and prunes unreachable template instances from schedule-fixed binaries at compile time to reduce binary size.
(4) On an Ascend NPU, TilingInfer achieves a 1.06× geometric-mean device-kernel speedup across 14 CANN operators and a 1.05× geometric-mean end-to-end speedup across six representative inference scenarios; it provides larger benefits in the performance-critical LLM decode phase, averaging a 1.10× end-to-end speedup across the two decode scenarios. On an NVIDIA GPU, it reduces a 542 MB FlashInfer archive by 88.8–95.5% across 17 model variants.

## 2 Background and Motivation

### 2.1 Graph IR and External Operator Libraries

A Graph Intermediate Representation (Graph IR) is a typed, directed dataflow graph: nodes encode inputs, attributes, operator calls, and outputs; edges carry tensors or scalars; and metadata records types and shapes. Framework IRs, such as PyTorch FX and TensorFlow graphs, preserve calls and arguments, while AI compiler IRs, such as TVM Relay/Relax, XLA HLO/StableHLO, and Inductor IR, normalize operations for optimization and lowering [3, 17, 21, 24, 28]. We focus on framework IR operator nodes because they define calls to external expert-written operator libraries.

With `torch.compile`, Dynamo extracts PyTorch operations from Python bytecode into an FX Graph [3, 23]. An FX operator node has $N = (\mathtt{op}, \mathtt{target}, \mathtt{args}, \mathtt{kwargs}, \mathtt{meta})$: `op`/`target` identify the call, `args`/`kwargs` hold actual arguments, and `meta` stores type and shape information. We use the following simplified Flash-Attention computation:

$$\mathrm{FA}(Q, K, V, M) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \tag{1}$$

where $Q \in \mathbb{R}^{B \times h_q \times S_q \times d_k}$, $K, V \in \mathbb{R}^{B \times h_{kv} \times S_{kv} \times d_k}$, and $M$ is an optional mask [8]. In Equation 1, $B$, $S_q$, and $S_{kv}$ may vary at runtime, whereas $h_q$, $h_{kv}$, $d_k$, and attention attributes are model-fixed. A corresponding FX node has `op` =

call_function, target = aten.scaled_dot_product_attention, args = (q,k,v), and kwargs = {attn_mask=None, dropout_p=0.0, is_causal=True}. For the concrete model instance used in our running example, the input FakeTensors q, k, and v have shapes $(s_B, 40, s_q, 128)$, $(s_B, 8, s_{kv}, 128)$, and $(s_B, 8, s_{kv}, 128)$, respectively, while the output FakeTensor in meta["val"] has shape $(s_B, 40, s_q, 128)$. Here, $s_B$, $s_q$, and $s_{kv}$ are SymInt symbols tracked by ShapeEnv, whereas 40, 8, and 128 are concrete integers.

*Example 2.1 (Scaled Dot-Product Attention Operator Signature).* The simplified C++ operator signature is:

```
scaled_dot_product_attention(query: Tensor, key: Tensor, value: Tensor, attn_mask:
    Optional[Tensor], dropout_p: float, is_causal: bool) -> Tensor
```

It binds q, k, and v to query, key, and value, and the keyword values to attn_mask, dropout_p, and is_causal; other optional parameters are omitted. Inside the C++ host implementation, every dimension is read uniformly through calls such as q->size(i). Such loads do not preserve whether the FX dimension was a concrete integer or a SymInt; the operator-library compiler therefore treats every loaded size as a runtime variable. This loss of graph-level shape information motivates its propagation into the C++ compilation flow.

## 2.2 Motivation of Automatic Model-Specific Specialization

In existing expert-written operator-library compilation pipelines, treating constant dimensions and operator attributes as variables creates two forms of waste: scalar and control overhead in schedule-generic kernels and unreachable template instances in schedule-fixed binaries. This subsection examines the opportunity for Graph-to-C++ constant propagation, the limitations of current compilation pipelines, and the key observation that most scheduling parameters are runtime invariants.

*2.2.1 Graph-to-C++ Constant Propagation.* DL models are partially dynamic: batch size and sequence lengths may vary at runtime, while dimensions and attributes such as head count are fixed by the model configuration. When a Python operator is invoked, its tensors and attributes cross the FFI boundary into the C++ host library, but their model-fixed status is not available to the separately compiled library. Figure 2 illustrates a fixed head count h=40 flowing into the host expression stages = (h > 32) ? 2 : 1. Propagating h=40 across the language boundary allows host-side constant propagation to infer the invariant schedule parameter stages=2. Making such schedule invariants available at compile time creates an opportunity to specialize schedule-generic kernels and discard incompatible pre-instantiated variants.

*2.2.2 Limitations of the Two Expert-Written Operator-Library Designs.*

*Bottleneck Analysis of Schedule-Generic Implementations.* Their kernels support complex scheduling strategies by receiving numerous schedule parameters from the host at runtime. Because the compiler cannot treat these parameters as constants, optimizations such as constant folding and loop simplification remain limited. Figure 3 illustrates the resulting overhead on Ascend.

Ascend combines Cube and Vector compute units with a programmable memory hierarchy. Kernels explicitly manage tile sizes, data layouts, and data movement across memory levels, while the Scalar unit computes addresses and loop bounds, handles control flow, and dispatches instructions. For example, when moving a tile from $Q_{GM}$ in global memory (GM) to $Q_{L1}$ in L1 memory, the Scalar unit determines its starting address, advances the loop, and accesses schedule parameters and scalar intermediates in GM or the Unified Buffer (UB). The representative profiles in Figure 4 show high

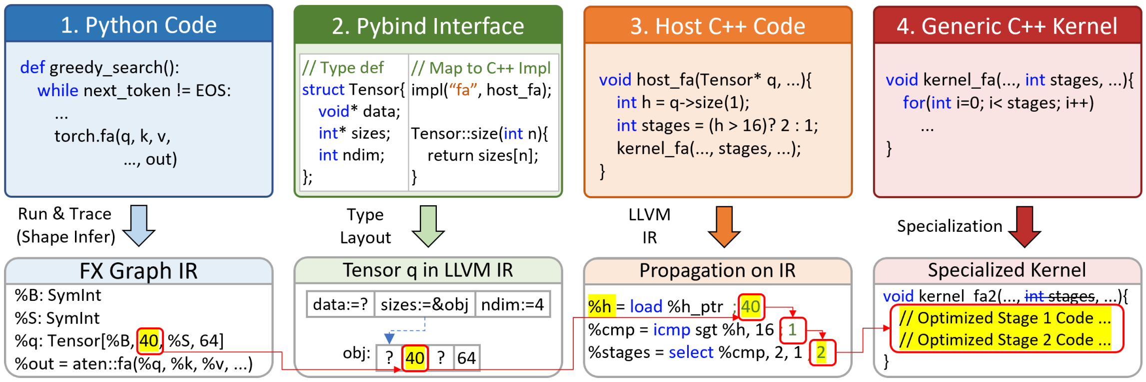



Fig. 2. A simplified Graph-to-C++ constant-propagation opportunity for Flash Attention. A model-fixed graph dimension (e.g., `q->size(1) = 40`) flows into C++ host code, determines the launch-site schedule parameter `stages = 2`, which can be used to specialize `kernel_fa` and generate an optimized kernel `kernel_fa2`.

Scalar ratios and nonzero I-Cache miss rates for several of these kernels. These measurements motivate device-side specialization to reduce the scalar workload and improve instruction locality.

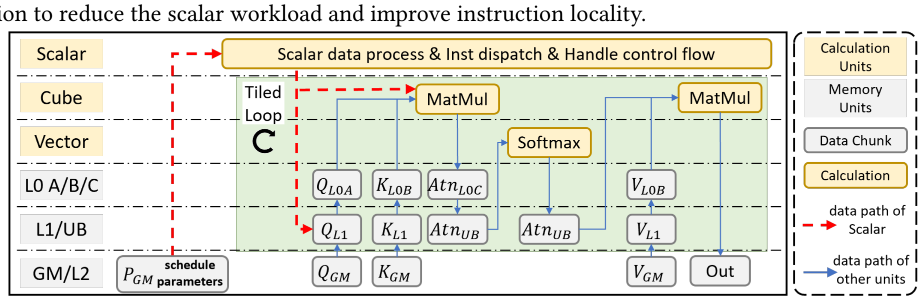



Fig. 3. Execution flow of Flash Attention kernel `kernel_fa` on the Ascend architecture. The **Scalar** unit acts as the central scheduler. It continuously performs scalar calculations and dispatches instructions based on runtime parameters. The heavy scalar workload required by generic kernels may lead to a scalar bottleneck.

*Code Bloat in Schedule-Fixed Implementations.* High-performance GPU libraries such as FlashInfer [37] rely on C++ template metaprogramming to enable aggressive compiler optimizations. To avoid runtime JIT overhead, such libraries pre-instantiate kernels for the Cartesian product of all template parameters. As described in Equation 1, FA's configuration space includes head dimensions ($d_k \in \{64, 128, 256\}$), mask types, GQA sizes, and position encodings. Combined with tile size configurations, this results in over 1000 binary instantiations, yet a specific model utilizes only ~10 kernels. This pre-instantiation strategy, while manageable on GPUs, is ill-suited for Ascend due to its more complex schedule parameters, forcing their Ascend counterparts to rely on schedule-generic kernels with limited compile-time optimization.

*2.2.3 Why Is Automatic Model-Specific Specialization Beneficial?* We further analyze the schedule parameters of key operators from the **Qwen2.5-1.5B** model, classifying them into *invariable parameters* (constants derived from model configuration) and *variable parameters* (runtime-dependent values such as batch size and sequence length). Figure 5 reveals that the overwhelming majority of kernel parameters are invariable. Propagating model-specific constants to resolve schedule parameters at compile time yields two benefits: (1) for schedule-generic implementations, it enables model-specific kernel specialization with aggressive constant folding, reducing scalar overhead and I-Cache misses; (2) for schedule-fixed implementations, it makes kernel dispatch paths deterministic, enabling precise pruning of unused binaries.

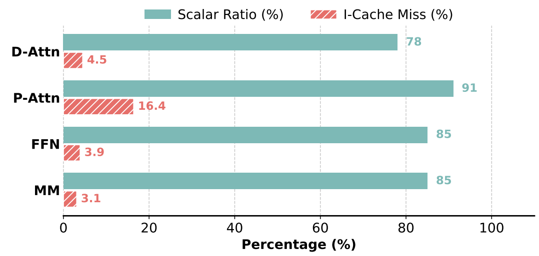



Fig. 4. Performance profiles of schedule-generic kernels, where lower values are better for both metrics: Decoding Attention (**D-Attn**), Prefill Attention (**P-Attn**), Feed Forward Networks (**FFN**), and MatMul (**MM**).

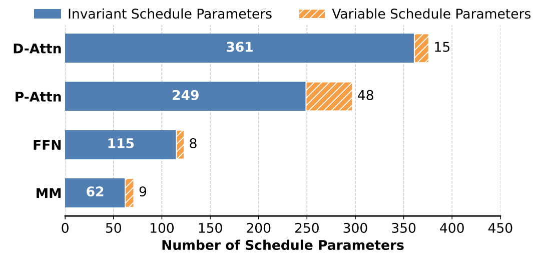



Fig. 5. Number of schedule parameters of representative kernels. The vast majority are runtime invariants (Invariable Params), validating the potential for specialization.

Besides, the further sensitivity study in §4.4 analyzes which schedule parameters tend to have the most significant impact on performance. It reveals that a small subset of parameters, which lie in the hot path of the kernel, are responsible for the majority of performance impact.

## 2.3 Challenges and Design Insights

Achieving automatic model-specific specialization requires transferring graph-level facts into C++ compilation and preserving them through host-side control flow. These steps present two challenges not directly addressed by existing static analyses.

*2.3.1 Challenge 1: Reconstructing Cross-Language Calling Contexts.* The first challenge is the Python/C++ semantic gap: Graph IR constants and symbolic dimensions must be mapped to the memory states of their underlying C++ objects before LLVM can use them as compile-time facts. This mapping is not a direct binding between arguments of graph IR node and C++ parameters. Graph IR records typed values and shapes, whereas composite C++ parameters such as `Tensor`, `List`, and `Optional` use nested object layouts; after lowering, an access such as `query->size(1)` becomes pointer arithmetic and `load` instructions. The analyzer must therefore reconstruct the corresponding base objects, pointers, field offsets, and field values, preserving concrete dimensions as constants and symbolic dimensions as unknown.

Existing cross-language analyses broadly follow two approaches: multilanguage abstract interpretation jointly analyzes programs on both sides of an FFI boundary [19], while specification-based methods summarize native behavior under a cross-language caller context [15]. The former would have to traverse the Python Graph Executor and FFI glue for dispatch, type conversion, object construction, and reference counting, which is expensive and largely irrelevant to graph-level constants. The latter assumes or derives a cross-language caller context but does not define how Graph IR metadata should materialize the field-sensitive C++ entry state required by LLVM.

*Insight: Type-directed calling-context construction.* DL operator interfaces use a limited set of parameter types, and their C++ interface layouts are stable within a backend. We can therefore define a context-update rule for each type and use Graph IR shapes and attributes to construct an accurate C++ operator calling context directly. This approach avoids analyzing FFI glue code; Table 2 formalizes the context-update rules used by TilingInfer.

*2.3.2 Challenge 2: Shape Checkers Leading to Early-Exit Paths.* The second challenge is preserving constant-propagation precision across shape-checker-induced early-exit paths without expensive path-sensitive analysis. Operator libraries enforce constraints for memory safety and correctness; for example, Flash Attention requires equal key and value

```
const status SUCCESS=0, ERROR=-1;
status check(int s1, int s2) {
  if(s1 != s2) return ERROR;
  return SUCCESS;
}
status set_t(int sk, int sv, int* t) {
  int e = check(sk, sv);
  if(e != SUCCESS) {
    print("error"); return e;
  }
  *t=32; // Tiling
  return SUCCESS;
}
```

Code 1. Early-exit path example.

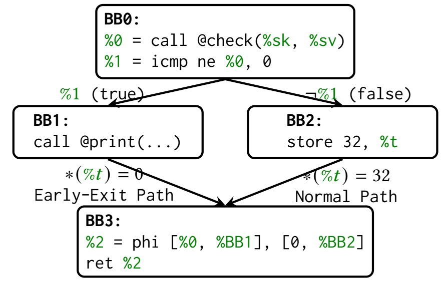



Fig. 6. Control Flow Graph (CFG) of set_t's LLVM IR. Each node is a basic block containing LLVM IR instructions. Edges are labeled with branch conditions.

sequence lengths ($S_k = S_v$). Because these dimensions may remain symbolic, the compiler cannot decide whether a check succeeds. An early-exit path then bypasses assignments performed on the normal path, and merging the two states can obscure constants.

In Code 1, check returns SUCCESS or ERROR, and set_t computes t=32 only after the check succeeds. The CFG in Figure 6 branches at **BB0**: **BB1** returns through the early-exit path without defining t, whereas **BB2** defines it before both paths reach **BB3**. If t initially holds 0, path-insensitive merging yields $\{0, 32\}$ rather than the normal-path constant 32. Enumerating all paths would avoid this pollution, but nested shape checkers make full path-sensitive analysis impractical.

*Insight: Conditional Return-Value Flow.* CRVFG-based normal-path reasoning traces only the conditional flow of status return values and compares them with configured success and error sets. In the example, the guard $\%1 = (\%0 \neq 0)$ being true implies that check's return value is not SUCCESS, so **BB0**→**BB1** is an early-exit edge; its false branch is the normal edge. Confirmed early-exit-edge states are excluded from normal-path merges, while unclassified edges are retained conservatively.

## 3 System Design of TilingInfer

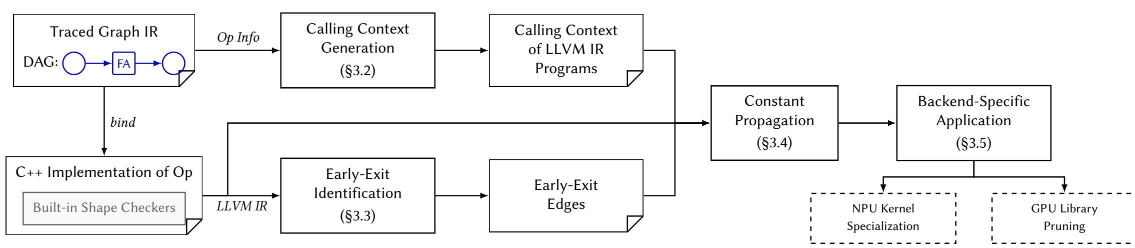



Fig. 7. The overall workflow of TilingInfer.

Figure 7 illustrates the workflow of TilingInfer. Here, an early-exit edge is the CFG edge that enters an early-exit path after a shape check fails. The framework takes the traced Graph IR containing operator invocation information with model-specific constants as input, and proceeds through four stages: **Context Generation** (§3.2), **Early-Exit Identification** (§3.3), **Constant Propagation** (§3.4), and **Kernel Specialization** (§3.5). We detail each stage in the following subsections.

Table 1. Abstract domains and definitions in TilingInfer.

| Domain | Notation | Domain | Notation |
|---|---|---|---|
| Top-level Variable | $v \in \mathcal{V}$ | Constant Value | $c \in \mathbb{Z} \cup \mathbb{R} \cup \mathcal{F}$ |
| Base Object | $b \in \mathcal{B}$ | Abstract Pointer | $p := \langle b, \hat{\delta} \rangle$ |
| Function Object | $f \in \mathcal{F}$ | Specialized Value | $sv := c \mid p \mid \top \mid \bot \in \mathcal{SV}$ |
| Concrete Offset | $\delta \in \mathbb{Z}$ | Environment | $\mathbb{E} := \mathcal{V} \mapsto \mathcal{SV}$ |
| Abstract Offset | $\hat{\delta} \in \mathbb{Z} \cup \{\bot\}$ | Store | $\mathbb{S} := \mathcal{O} \mapsto \mathcal{SV}$ |
| Memory Location | $o := \langle b, \delta \rangle \in \mathcal{O}$ | Calling Context | $\mathbb{C} := (\mathbb{E}, \mathbb{S})$ |

## 3.1 Definitions

To formally describe the analysis of TilingInfer, we first define the abstract domains and notations used in our framework, as summarized in Table 1.

*Memory Model and Variables.* We distinguish between SSA values and memory locations:

- *Top-level variables* ($v \in \mathcal{V}$) correspond to virtual registers in the LLVM IR.
- *Base objects* ($b \in \mathcal{B}$) represent the abstract base addresses of memory blocks created at allocation sites (e.g., `alloca` instructions, global variables, or heap allocations).
- *Memory locations* ($o \in \mathcal{O}$) represent specific fields within allocated objects. A location is defined as a pair $\langle b, \delta \rangle$, where $\delta \in \mathbb{Z}$ denotes a concrete byte offset relative to the base address $b$.

*Abstract Values and Lattice.* The analysis tracks the state of variables using *Specialized Values* ($sv \in \mathcal{SV}$). The domain $\mathcal{SV}$ forms a lattice comprising the following elements:

- *Top ($\top$) and Bottom ($\bot$):* $\top$ represents an *undefined* or *uninitialized* state (the lattice top, indicating no information), while $\bot$ represents a generic *variable* state where the value cannot be statically determined (the lattice bottom, indicating over-approximation).
- *Concrete Constants ($c$):* These represent precise, known values. This category includes integers ($\in \mathbb{Z}$), floating-point numbers ($\in \mathbb{R}$), and *Function Objects* ($f \in \mathcal{F}$), which represent references to functions targeted by direct or indirect calls.
- *Abstract Pointers ($p$):* Defined structurally as $\langle b, \hat{\delta} \rangle$. A pointer $p$ represents an address computed by adding an *Abstract Offset* $\hat{\delta}$ to a base object $b$. Here, $\hat{\delta} \in \mathbb{Z} \cup \{\bot\}$ allows the analysis to track precise offsets when possible, or degrade to $\bot$ if the offset is variable (e.g., a runtime index).

*Context and State.* The program state and interprocedural context are captured by the following mappings:

- The *Environment* $\mathbb{E} : \mathcal{V} \mapsto \mathcal{SV}$ maps top-level virtual registers to their abstract values.
- The *Store* $\mathbb{S} : \mathcal{O} \mapsto \mathcal{SV}$ maps memory locations to the values stored therein. By mapping from pairs of $\langle b, \delta \rangle$, the store naturally supports field-sensitive tracking of structure fields and array elements.
- The *Calling Context* $\mathbb{C} := (\mathbb{E}, \mathbb{S})$ encapsulates the analysis state (environment and store) at the entry basic block of the current function. This context is determined by the caller and propagates constraints across function boundaries during interprocedural analysis.

## 3.2 Calling Context Generation for C++

Figure 8 illustrates the overall workflow of context generation in TilingInfer. The process takes a Graph IR node $N$ representing an operator invocation as input, and produces the initial calling context $\mathbb{C}_{init}$ for the corresponding C++ entry function. This process bridges the representation and compilation boundary between Graph IR metadata and the abstract

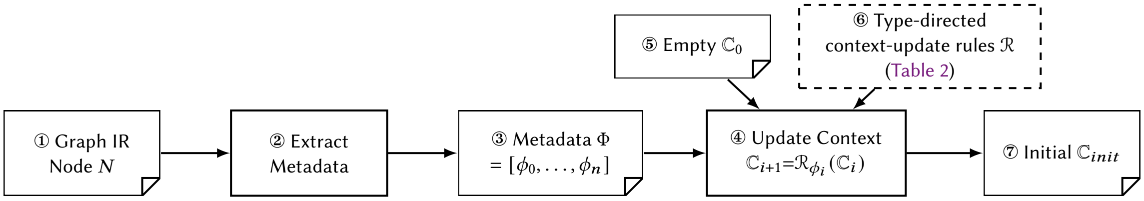


Fig. 8. Workflow of calling-context generation. ①–③: extraction parses the Graph IR node into typed metadata $\Phi$. ④–⑦: the five type-directed context-update rules $\mathcal{R}$ in Table 2 map $\Phi$ to the abstract C++ calling context $\mathbb{C}_{init}$.

C++ entry state required by LLVM analysis through two phases: (1) **Extraction** (①–③), which parses the IR node $N$ to generate structured metadata $\Phi$; and (2) **Construction** (④–⑦), which applies the context-update rules $\mathcal{R}$ to transform the empty context $\mathbb{C}_0$ into the initial calling context $\mathbb{C}_{init}$. Here, $\mathcal{R} = \{\mathcal{R}_{\text{SCALAR}}, \mathcal{R}_{\text{STRING}}, \mathcal{R}_{\text{TENSOR}}, \mathcal{R}_{\text{LIST}}, \mathcal{R}_{\text{OPTIONAL}}\}$ is the family of five type-directed context-update rules defined in Table 2. Each rule maps one typed metadata item to updates of the abstract environment and store, thereby materializing the C++ entry-state fields accessed by the target operator.

*3.2.1 Graph IR Node and Metadata Extraction.* The Graph IR is a functional intermediate representation based on the ATen operator set, organized as a directed acyclic graph where each node represents a logical ATen operator invocation. We formally define a Graph IR node as:

$$N = (\mathsf{op}, \mathit{In}, \mathit{Out}, \sigma) \tag{2}$$

where $\mathsf{op}$ is the ATen operator identifier (e.g., `aten.scaled_dot_product_attention`, `aten.linear`), $\mathit{In} = [x_1, \ldots, x_m]$ is the ordered input list, $\mathit{Out} = [y_1, \ldots, y_k]$ is the ordered output list, and $\sigma$ is the shape environment. Each input $x_i$ is either a tensor produced by another node or an attribute, such as an integer, a floating-point value, or an array. The type of each input and output is directly represented by the corresponding metadata item $\phi_i$ in the context metadata $\Phi$, which is derived from the operator signature. The shape environment $\sigma$ maps each tensor value $v$ to its type and shape information:

$$\sigma(v) = (\mathtt{dtype}(v), \mathtt{shape}(v)) \tag{3}$$

where $\mathtt{shape}(v) = [d_1, \ldots, d_r]$ is a dimension list with each $d_i$ being either a concrete integer or a symbolic integer (`SymInt`) representing a runtime-determined dimension. This design enables the Graph IR to capture dynamic shape information symbolically.

ATen operators have strictly defined signatures that specify the types of inputs and outputs. Each parameter type is directly mapped to a metadata variant $\phi$ (e.g., Tensor, Scalar, List, Optional) without introducing a separate type variable.

TilingInfer extracts ② the *context metadata* $\Phi$ ③ by aligning the Graph IR node's arguments with the operator's formal parameters. For each parameter in the signature, if an explicit argument is provided, TilingInfer captures its value or shape information as a metadata item $\phi_i$ according to the parameter's type; if the argument is omitted and a default value exists, the default is used.

The context metadata $\Phi = [\phi_0, \phi_1, \ldots, \phi_n]$ is a sequence where each $\phi_i$ is a tagged union capturing the type and value information:

$$\phi ::= \text{SCALAR}(c) \mid \text{STRING}(s) \mid \text{TENSOR}(\vec{d}) \mid \text{LIST}\langle\phi'\rangle(\vec{e}) \mid \text{OPTIONAL}\langle\phi'\rangle(v) \tag{4}$$

Each parameter type constructor corresponds to the semantic meaning in the C++ operator interface:

- Scalar($c$): Represents a scalar parameter of type Int or Float, where $c$ is the concrete value.

- $\textsc{String}(s)$: Represents a string parameter, where $s$ is the string content.
- $\textsc{Tensor}(\overrightarrow{d})$: Represents a tensor parameter, where $\overrightarrow{d} = [d_1, \ldots, d_k]$ is the dimension list with each $d_i$ being either a concrete constant or a symbolic expression.
- $\textsc{List}\langle\phi'\rangle(\overrightarrow{e})$: Represents a list parameter of element type $\phi'$, where $\overrightarrow{e}$ is the sequence of elements.
- $\textsc{Optional}\langle\phi'\rangle(v)$: Represents an optional parameter, where $v$ is either a concrete value of type $\phi'$ (present) or the special value None (absent).

*Example 3.1.* Consider the `aten.scaled_dot_product_attention` invocation and the simplified C++ operator signature introduced in example 2.1. The Graph IR node is $N = (\texttt{aten.scaled\_dot\_product\_attention}, \mathit{In}, \mathit{Out}, \sigma)$, where $\mathit{In} = [\mathsf{q}, \mathsf{k}, \mathsf{v}, \mathsf{None}, 0.0, 1]$ follows the signature order, $\mathit{Out} = [\mathsf{o}]$, and 1 encodes `is_causal=True`. The shape environment records:

$$\sigma(\mathsf{q}) = (\texttt{dtype}(\mathsf{q}), [s_B, 40, s_q, 128]), \qquad \sigma(\mathsf{k}) = (\texttt{dtype}(\mathsf{k}), [s_B, 8, s_{kv}, 128]),$$
$$\sigma(\mathsf{v}) = (\texttt{dtype}(\mathsf{v}), [s_B, 8, s_{kv}, 128]), \qquad \sigma(\mathsf{o}) = (\texttt{dtype}(\mathsf{o}), [s_B, 40, s_q, 128]).$$

Thus, $s_B$, $s_q$, and $s_{kv}$ remain symbolic, while 40, 8, and 128 are concrete. The extracted metadata $\Phi$ is:

$$\begin{aligned}\Phi = [&\textsc{Tensor}([s_B, 40, s_q, 128]), \textsc{Tensor}([s_B, 8, s_{kv}, 128]),\\ &\textsc{Tensor}([s_B, 8, s_{kv}, 128]), \textsc{Optional}\langle\textsc{Tensor}\rangle(\text{None}),\\ &\textsc{Scalar}(0.0), \textsc{Scalar}(1)].\end{aligned}$$

*3.2.2 Context Construction from Metadata.* The goal of the construction phase is to synthesize the initial C++ calling context $\mathbb{C}_{init} = (\mathbb{E}, \mathbb{S})$ from the metadata $\Phi$. Starting with an empty context $\mathbb{C}_0$ ⑤, TilingInfer iterates over the metadata items $\phi_i$ and sequentially updates ④ the abstract state: for $\tau_i = \text{type}(\phi_i)$, we write $\mathcal{R}_{\phi_i}(\mathbb{C}) := \mathcal{R}_{\tau_i}(\mathbb{C}, \phi_i)$ for the instance of the matching type-directed context-update rule.

$$\mathbb{C}_{init} = \mathcal{R}_{\phi_n} \circ \cdots \circ \mathcal{R}_{\phi_1} \circ \mathcal{R}_{\phi_0}(\mathbb{C}_0) \tag{5}$$

Thus, each $\mathcal{R}_{\phi_i}$ in the equation is an application of exactly one of the five context-update rules in $\mathcal{R}$ shown in Table 2, transforming the high-level metadata into the low-level memory representation.

*3.2.3 Context-Update Rules for Metadata Types.* Table 2 presents the five context-update rules, one for each supported metadata type.

For Scalar and String types, the corresponding context-update rule directly stores the value in the environment. For composite types like Tensor and List, the context-update rules perform *deep reconstruction*: allocating base objects for internal structures and linking them via pointers in the store. We use a helper function $\alpha(x)$ that returns $x$ if $x$ is a concrete constant, or $\perp$ if $x$ is a symbolic expression, where $\perp$ denotes *unknown-but-consistent* values fixed at runtime but not statically determinable.

Table 2. The five type-directed context-update rules $\mathcal{R}$ for constructing the abstract C++ entry state. **Definitions:** alloc(`T`) creates an abstract base object with the validated fields accessed through type `T`. Process($type, v, \ell$) recursively populates the store at location $\ell$ with value $v$ of type $type$. $\alpha(x)$ returns $x$ if concrete, or $\perp$ if symbolic.

| Context-Update Rule | Abstract Fields | Environment Update | Store Update |
|---|---|---|---|
| $\mathcal{R}_{\text{Scalar}}$<br>$\phi = \text{Scalar}(c)$ | `int64_t c;`<br>`double c;` | $v \mapsto \alpha(c)$ | None |
| $\mathcal{R}_{\text{String}}$<br>$\phi = \text{String}(s)$ | `struct {`<br>`size_t len;`<br>`char* data;`<br>`}` | $\ell = \text{alloc}(\texttt{str\_view})$<br>$\ell_{buf} = \text{alloc}(\texttt{char}[\lvert s\rvert])$<br>$v \mapsto \langle \ell, 0\rangle$ | $\langle \ell, \delta_{\texttt{len}}\rangle \mapsto \lvert s\rvert$<br>$\langle \ell, \delta_{\texttt{data}}\rangle \mapsto \langle \ell_{buf}, 0\rangle$<br>$\forall i.\ \langle \ell_{buf}, i\rangle \mapsto s[i]$ |
| $\mathcal{R}_{\text{Tensor}}$<br>$\phi = \text{Tensor}(\overrightarrow{d})$<br>$\overrightarrow{d} = [d_1 \ldots d_k]$ | `struct {`<br>`int64_t* sizes;`<br>`int64_t ndim;`<br>`}` | $\ell = \text{alloc}(\texttt{Tensor})$<br>$\ell_{sz} = \text{alloc}(\texttt{int64\_t}[k])$<br>$v \mapsto \langle \ell, 0\rangle$ | $\langle \ell, \delta_{\texttt{ndim}}\rangle \mapsto k$<br>$\langle \ell, \delta_{\texttt{sizes}}\rangle \mapsto \langle \ell_{sz}, 0\rangle$<br>$\forall i.\ \langle \ell_{sz}, i \cdot \text{sz}_{\texttt{int64\_t}}\rangle \mapsto \alpha(d_i)$ |
| $\mathcal{R}_{\text{List}}$<br>$\phi = \text{List}\langle type\rangle(\overrightarrow{e})$<br>$\overrightarrow{e} = [e_1 \ldots e_n]$ | `struct {`<br>`size_t size;`<br>`T* data;`<br>`}` | $\ell = \text{alloc}(\texttt{List}\langle type\rangle)$<br>$\ell_{buf} = \text{alloc}(type[n])$<br>$v \mapsto \langle \ell, 0\rangle$ | $\langle \ell, \delta_{\texttt{size}}\rangle \mapsto n$<br>$\langle \ell, \delta_{\texttt{data}}\rangle \mapsto \langle \ell_{buf}, 0\rangle$<br>$\forall i.\ \text{Process}(type, e_i, \langle \ell_{buf}, i \cdot \text{sz}_{type}\rangle)$ |
| $\mathcal{R}_{\text{Optional}}$<br>$\phi = \text{Optional}\langle type\rangle(u)$ | `struct {`<br>`bool has_val;`<br>`T* val;`<br>`}` | $\ell = \text{alloc}(\texttt{Optional}\langle type\rangle)$<br>if $u \neq$ None :<br>$\ell_{obj} = \text{alloc}(type)$<br>$v \mapsto \langle \ell, 0\rangle$ | $\langle \ell, \delta_{\texttt{has\_val}}\rangle \mapsto (u \neq \text{None})$<br>if $u \neq$ None :<br>$\langle \ell, \delta_{\texttt{val}}\rangle \mapsto \langle \ell_{obj}, 0\rangle$<br>$\text{Process}(type, u, \langle \ell_{obj}, 0\rangle)$ |

The recursive helper function Process($type, v, \ell$) populates the store at location $\ell$ with value $v$ of type $type$. For example, the $\mathcal{R}_{\text{Tensor}}$ context-update rule is defined as:

$$\begin{aligned}
&\textbf{Process}(\texttt{Tensor}, [d_1, \ldots, d_k], \ell) :\\
&\quad \ell_{sz} = \text{alloc}(\texttt{int}[k])\\
&\quad \mathbb{S}[\langle \ell, \delta_{\texttt{sizes}}\rangle] = \langle \ell_{sz}, 0\rangle;\ \mathbb{S}[\langle \ell, \delta_{\texttt{ndim}}\rangle] = k\\
&\quad \textbf{for } i = 1 \textbf{ to } k \textbf{ do } \mathbb{S}[\langle \ell_{sz}, (i-1) \cdot \text{sz}_{\texttt{int}}\rangle] = \alpha(d_i)
\end{aligned}$$

*Example 3.2.* Consider a `Tensor` parameter `query` with shape $[s_1, 40]$ (symbolic $s_1$, concrete 40). After applying the $\mathcal{R}_{\text{Tensor}}$ context-update rule:

$$\mathbb{E} = \{\texttt{query} \mapsto \langle \ell, 0\rangle\}$$

$$\mathbb{S} = \{\langle \ell, \delta_{\texttt{sizes}}\rangle \mapsto \langle \ell_{sz}, 0\rangle,\ \langle \ell_{sz}, 0\rangle \mapsto \perp,\ \langle \ell_{sz}, \text{sz}_{\texttt{int}}\rangle \mapsto 40\}$$

The environment maps `query` to a pointer, and the store captures the internal structure with $\perp$ for symbolic dimension $s_1$ and 40 for the concrete dimension.

**Completeness.** The context-update rules cover all five core parameter types (Scalar, String, Tensor, List, Optional) in the PyTorch operator interface. Recursive applications of the context-update rules enable complex compositions like Optional⟨List⟩, ensuring the approach is generic across standard operator schemas.

### 3.3 Early-Exit Edge Identification

Production operator libraries manage execution status through compile-time constant integer status codes. We define two sets: $\mathcal{S}$ (success codes, e.g., $\{0\}$) and $\mathcal{E}$ (error codes, e.g., $\{-1\}$). The early-exit edge identification process consists of three steps: (1) constructing a Conditional Return-Value Flow Graph (CRVFG) to capture the relationship between shape-checker conditions and return values; (2) determining error states through constant comparison; and (3) mapping identified error conditions to control-flow edges.

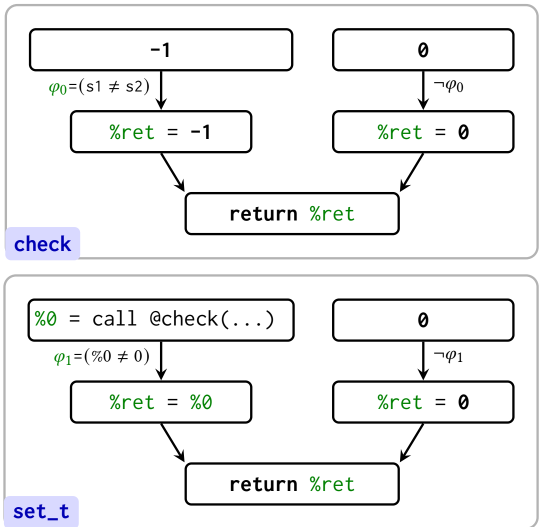



Fig. 9. Return-Value Flow Graphs for check and `set_t` functions. For check: direct error return when $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$ holds, the return value is $-1 \in \mathcal{E}$. For `set_t`: propagated error when $\varphi_1 = (\%0 \neq 0)$ holds, the error code from check flows to the return.

**Algorithm 1. Early-Exit Edge Identification**

**Require:** Function $F$, Success set $\mathcal{S}$, Error set $\mathcal{E}$
**Ensure:** Set of early-exit edges $E_{exit}$

$E_{exit} \leftarrow \emptyset$
$inst \leftarrow \texttt{GetReturnInst}(F)$
$v_{ret} \leftarrow \texttt{GetReturnValue}(inst)$
$P_{flow} \leftarrow \texttt{BuildCRVFG}(F, v_{ret})$
**for all** $path \in P_{flow}$ **do**
  $\varphi \leftarrow \texttt{GetCondition}(path)$
  $v \leftarrow \texttt{GetSourceValue}(path)$
  **if** isError$(v, \varphi)$ **then**
    $\varphi_{key} \leftarrow \texttt{GetKeyClause}(\varphi)$
    $B_{src} \leftarrow \texttt{GetBasicBlock}(\varphi_{key})$
    $B_{dst} \leftarrow \texttt{GetSuccessor}(B_{src}, \varphi_{key})$
    $\texttt{MarkEarlyExitEdge}(B_{src} \rightarrow B_{dst})$
    $E_{exit} \leftarrow E_{exit} \cup \{B_{src} \rightarrow B_{dst}\}$
  **end if**
**end for**
**return** $E_{exit}$

*3.3.1 Conditional Return-Value Flow Graph Construction.* The first step constructs a *Conditional Return-Value Flow Graph* (CRVFG) to capture the relationship between shape-checker conditions and return values. The CRVFG is a directed graph $G = (V, E)$ where:

- Each node $v \in V$ represents a definition of the return value (e.g., a `phi` node or a constant assignment).
- Each edge $e = (v_s, v_d, \varphi) \in E$ represents a data-flow dependency from $v_s$ to $v_d$ under branch predicate $\varphi$.

The CRVFG is constructed *on demand* and *intraprocedurally*:

(1) **On-Demand:** We construct the graph only for functions containing shape checkers, avoiding overhead for irrelevant functions.
(2) **Return-Value Focus:** We exclusively track the value flow of return values, which in SSA-form IR are top-level variables, yielding compact graphs.

Figure 9 illustrates the CRVFGs for both the check and `set_t` functions in Code 1. In check, the condition $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$ guards the path where the return value is $-1 \in \mathcal{E}$, directly identifying an early-exit path. In `set_t`, the condition $\varphi_1 = (\%0 \neq 0)$ guards the path where the error code from check flows to the return. The CRVFG thus characterizes the conditions guarding each value-flow path, enabling us to determine which paths lead to error states.

*3.3.2 Error Status Determination.* The second step determines whether a return value indicates an error state under a given condition.

Operator shape checkers fall into two categories based on the source of their return values: (1) *Direct-constant-return checkers* (e.g., the check function) that directly return a constant value, typically an error code, when validation fails; and (2) *Variable-return checkers* (e.g., the `set_t` function) that return a variable, typically the return value of another function call.

For a value-flow path with return value $v$ and governing condition $\varphi$, we classify it as an early-exit path if and only if:

$$\text{isError}(v, \varphi) = \begin{cases} \text{true} & \text{if } \varphi \Rightarrow (v \notin \mathcal{S}) \lor \varphi \Rightarrow (v \in \mathcal{E}) \\ \text{false} & \text{otherwise} \end{cases} \tag{6}$$

Intuitively, an early-exit path is identified when $\varphi$ implies that $v$ is either not a success code or is an error code.

For *Direct-constant-return checkers*, since $v$ is a constant, we directly test whether $v \notin \mathcal{S}$ or $v \in \mathcal{E}$ holds. For *Variable-return checkers*, $\varphi$ contains comparisons against constants (e.g., $v = -1$ or $v \neq 0$), where the constants are typically error or success codes. We compute the possible value set of $v$ from $\varphi$, then test whether all possible values are outside $\mathcal{S}$ or belong to $\mathcal{E}$. Both cases require only constant comparison and set-membership operations computable in $O(1)$ time, without invoking an SMT solver.

Figure 9 illustrates this process. For `check`, under $\varphi_0 = (\texttt{s1} \neq \texttt{s2})$, the return value is the constant $-1$. Since $-1 \in \mathcal{E}$, $\text{isError}(-1, \varphi_0) = \text{true}$. For `set_t`, under $\varphi_1 = (\%0 \neq 0)$, the return value is $\%0$, which is returned by `check`. From $\varphi_1$, we infer that $\%0 \in \mathbb{Z} \setminus \{0\}$. Since $\mathcal{S} = \{0\}$, all possible values are outside $\mathcal{S}$, so $\text{isError}(\%0, \varphi_1) = \text{true}$.

*3.3.3 Early-Exit Edge Identification on CFG.* The third step maps identified early-exit conditions to CFG edges, as presented in Algorithm 1. For each value-flow path in the CRVFG, where a path represents a complete flow from a source (a call instruction or constant) to the return instruction, the algorithm extracts the guard condition $\varphi$ and source value $v$, then invokes $\text{isError}(v, \varphi)$ to determine whether the path is an early-exit path. If an early-exit path is identified, the algorithm extracts the key clause $\varphi_{key}$ from $\varphi$ via `GetKeyClause`. This operation returns the rightmost sub-clause (e.g., $\varphi = (b_1 = b_2) \land (\texttt{s1} \neq \texttt{s2})$ yields $\varphi_{key} = (\texttt{s1} \neq \texttt{s2})$), because the rightmost clause determines the error status under short-circuit evaluation. The algorithm obtains the basic block $B_{src}$ containing $\varphi_{key}$ via $\texttt{GetBasicBlock}(\varphi_{key})$ and the successor block $B_{dst}$ via $\texttt{GetSuccessor}(B_{src}, \varphi_{key})$, then marks the CFG edge $B_{src} \rightarrow B_{dst}$ as an early-exit edge.

*3.3.4 Inter-Procedural Recursion.* When a function containing shape checkers has a return value derived from a callee, inter-procedural recursion is required to identify early-exit paths across functions. Two scenarios trigger recursive analysis of a callee:

(1) If the source value $v$ of a value-flow path is the return value of a call instruction, the callee is recursively analyzed.
(2) If a clause in the guard condition $\varphi$ checks whether the return value of a call instruction belongs to $\mathcal{E}$ or $\mathcal{S}$ (e.g., $\varphi = (\%0 \neq 0)$ where $\%0$ is a call return), the callee is recursively analyzed. This case corresponds to common status-propagation patterns in production code (e.g., `if (x == ERROR) return ERROR`), where `x` is the return value of a callee.

### 3.4 Inter-Procedural Constant Propagation for Host-Side Programs

The propagation stage takes the initial calling context $\mathbb{C}_{init}$ from §3.2 and the early-exit edges $E_{exit}$ from §3.3 as inputs, and propagates constants through the host-side program to kernel launch sites. The analysis maintains a symbolic environment $\mathbb{E}$ and a flow-sensitive memory store $\mathbb{S}$, performing value propagation over the domain defined in Table 1.

*Initialization.* The analysis begins with $\mathbb{C}_{init} = (\mathbb{E}, \mathbb{S})$ constructed from Graph IR metadata, which provides concrete values for tensor dimensions and operator parameters. These constants seed propagation: when the analysis loads from a memory location initialized with a concrete value, that value flows into the corresponding virtual register.

*Propagation and Edge Liveness.* During forward propagation, conditional branches determine edge liveness. If a branch condition evaluates to a constant, the non-taken edge is marked `Inactive`; otherwise, both edges remain `Active`. At control-flow merge points, incoming stores from all `Active` predecessors are combined using standard lattice joins.

*Early-Exit Integration.* A key precision enhancement comes from integrating early-exit edge information. Any CFG edge in $E_{exit}$ is treated as `Inactive` during context merging. This excludes early-exit paths from the analysis state, preventing the pollution of propagated constants by imprecise values from unreachable branches. For example, if a shape checker returns an error code when dimensions mismatch, but the initial context guarantees matching dimensions, the error path is pruned and its state does not affect downstream analysis.

### 3.5 Kernel Specialization and Library Pruning

Based on the context derived from inter-procedural constant propagation, TilingInfer performs two key optimizations: schedule-generic kernel specialization and pruning redundant template instantiations from schedule-fixed operator libraries.

For schedule-generic kernels, TilingInfer substitutes kernel variables with the concrete schedule parameters identified by the host-side analysis. Then TilingInfer employs a sequence of optimization passes, specifically `IPSCCP`, `Loop Unrolling`, and `CFG Simplification`. These passes exploit the newly introduced constants to fold computations and simplify control structures, generating a highly optimized kernel binary.

For schedule-fixed libraries, TilingInfer reduces code size by leveraging path liveness information from the host analysis. During constant propagation, we identify `Dead Paths`, i.e., execution branches where guard conditions evaluate to false based on deterministic constants. If a specific template function instance is invoked only on these dead paths, TilingInfer determines it is unreachable at runtime and removes the instance entirely. This selective pruning ensures that only potentially active operator instances are retained in the final binary.

## 4 Evaluation

Our evaluation addresses the following research questions: **RQ1:** Can TilingInfer enhance performance across diverse operators and neural networks that are based on schedule-generic expert-written libraries? **RQ2:** Can TilingInfer effectively infer more invariant schedule parameters with reasonable compilation overhead? **RQ3:** Can TilingInfer effectively eliminate redundant code from schedule-fixed expert-written operator libraries? Beyond these questions, we evaluate detailed performance metrics and improvements under different shape configurations.

### 4.1 Experimental Setup

*Hardware and Software Platforms* We use two platforms: (1) **NPU Platform**—Ascend 910B1 NPU (64GB HBM) with Intel Core i7-8700 CPU and 64GB DRAM, running CANN 8.0.RC1, LLVM 15.0.5, and PyTorch 2.6.0. The end-to-end model experiments additionally use Transformers 4.57.6 and TorchRec 1.2.0 (reported metrics are averaged over 90 runs after 10 warm-up iterations); (2) **GPU Platform**—NVIDIA RTX 3090 (24GB VRAM) with Intel Xeon Gold 6348 CPU, running CUDA 12.2, PyTorch 2.6.0, vLLM 0.9.1, and FlashInfer v0.2.

*Benchmarks* For the **NPU platform**, we evaluate individual operator latency and end-to-end model inference. The operator benchmark covers operators from LLMs, Computer Vision, and recommendation models, with emphasis on complex fused operators such as `FlashAttention`, `FeedForward`, and `MatMul` that feature intensive workloads and numerous schedule parameters. Table 3 summarizes the categorization and configurations of the evaluated operators. For end-to-end evaluation, we evaluate six end-to-end scenarios comprise DeepFM recommendation inference [12],

Table 3. Overview of dynamic shape operator benchmarks. **Symbols:** $T$: total tokens, $B$: batch size, $P$: page size, $N_P$: number of KV-cache pages, $M_P$: maximum pages per request, $N_Q/N_{KV}$: query/key–value head counts, $D$: model dimension, $D_{head}$: head dimension ($D/N_Q$), $K$: matrix reduction dimension, $C$: channels, $E$: experts, and $H/W$: image height/width. **Aux. Input**: indexing or geometric metadata. **Configurations:** Unless otherwise stated, we fix static dimensions based on representative models: $D = 4096$, $D_{head} = 128$, $N_Q = 32$, $N_{KV} = 32$ for Llama2-7B; $E = 8$ for Mixture-of-Experts (MoE) layer; $C = 512$ for CV (e.g., ResNet-50); and $D = 128$ for RecSys. **Note:** Operators labeled **MM**, **FFN**, and **RMS** share identical kernel implementations across Prefill and Decoding phases, differing only in the leading dynamic dimension ($T$ vs. $B$). **Conv2d** is lowered to `im2col` followed by MatMul and is therefore reported as **MM**. Unless otherwise stated, each operator-level result is the geometric mean over five shapes sampled from the listed dynamic range; all methods use the same samples.

| Domain | Phase / Scenario | Operator | Abbr. | Main Layout | Aux. Input | Dynamic Range |
|---|---|---|---|---|---|---|
| **LLM** *Llama 2* *Qwen 2* *DeepSeek V3* | Prefill Phase (Ragged Input) | FlashAttention | **P-Attn** | $Q: [T, N_Q, D_{head}]$ $K, V: [T, N_{KV}, D_{head}]$ | `Offsets[B+1]` | $T \in [256, 8k]$ $B \in [1, 8]$ |
| | | FusedFFN | **FFN** | $[T, D]$ | - | $T \in [256, 8k]$ |
| | | RmsNorm | **RMS** | $[T, D]$ | - | $T \in [256, 8k]$ |
| | | MatMul | **MM** | $[T, K]$ | - | $T \in [256, 8k]$ |
| | Decoding Phase (Paged KV) | PagedAttention | **D-Attn** | $Q: [B, 1, N_Q, D_{head}]$ $K, V: [N_P, P, N_{KV}, D_{head}]$ | `BlockTbl[B, `$M_P$`]` `Offsets[B+1]` | $T \in [256, 8k]$ $B \in [1, 8]$ |
| | | FusedFFN | **FFN** | $[B, D]$ | - | $B \in [1, 8]$ |
| | | RmsNorm | **RMS** | $[B, D]$ | - | $B \in [1, 8]$ |
| | | MatMul | **MM** | $[B, K]$ | - | $B \in [1, 8]$ |
| | MoE Layer (Sparse Activation) | MoeGating | **Gate** | $[T, E]$ | - | $T \in [256, 8k]$ |
| | | GroupedMatMul | **GMM** | $[T, D]$ | `GroupOffsets[E+1]` | $T \in [256, 8k]$ |
| | | Sinkhorn | **Sink** | $[T, E]$ | - | $E \in [8, 64]$ |
| | | MoeTokenPermute | **Perm** | $[T, D]$ | `Indices[T]` | $T \in [256, 8k]$ |
| **CV** *ResNet,* *YOLOv5* | General Inference (Standard Batch) | Conv2d(img2col) | **MM** | $[B, C, H, W]$ | - | $B \in [1, 8]$ |
| | | GroupNorm | **GN** | $[B, C, H, W]$ | - | $B \in [1, 8]$ |
| | | AvgPool | **Pool** | $[B, C, D, H, W]$ | - | $B \in [1, 8]$ |
| | | ElementWise Pow | **Pow** | $[B, C, H, W]$ | - | $B \in [1, 8]$ |
| | Object Detection (Dyn. Resolution) | Upsample | **Upsample** | $[B, C, H, W]$ | - | $H, W \in [224, 1024]$ |
| | | BatchNorm | **BN** | $[B, C, H, W]$ | - | $B \in [1, 8]$ |
| **RecSys** | Feature Interaction (Sparse Update) | ScatterElements | **Scat** | $[T, D]$ | `Indices[T]` | $T \in [128, 1024]$ |

Table 4. End-to-end model benchmark configurations. $B$ is the number of concurrent requests, $S$ is the HSTU interaction-sequence or LLM prompt length, and $L_{\text{KV}}$ is the number of cached tokens visible to each decode request.

| **Scenario** | **Deployment stack** | **Fixed configuration** | **Dynamic Range in Figure 11** | **Shape in Figure 12** |
|---|---|---|---|---|
| DeepFM | `torch_npu` + TorchRec | 20 sparse fields, 13 dense features, $D_{\text{emb}} = 64$, 10 dense blocks | $B \in \{4, 16, 32, 64, 128\}$ | $B = 64$ |
| HSTU | `torch_npu` + TorchRec | $B = 2$, $D = 512$, $N_H = 8$, $D_H = 64$, 8 HSTU blocks; no interaction-sequence cache | $S \in \{32, 64, 128, 256, 512, 1024\}$ | $B = 2,\ S = 256$ |
| LLaMA-3 1B Prefill | `torch_npu` + Transformers | $B = 2$, $D = 512$, $N_H = 8$, $D_H = 64$, 16 decoder blocks | $S \in \{32, 64, 128, 256, 512, 1024\}$ | $B = 2,\ S = 256$ |
| LLaMA-3 1B Decode | `torch_npu` + Transformers | one-token query, $D = 512$, $N_H = 8$, $D_H = 64$, paged KV cache | $B = 2$, $L_{\text{KV}} \in \{32, 64, 128, 256, 512, 1024\}$ | $B = 16,\ L_{\text{KV}} = 256$ |
| Qwen-3.5 1B Prefill | `torch_npu` + Transformers | $B = 2$, $D = 640$, $N_H = 8$, $D_H = 80$, 16 decoder blocks | $S \in \{32, 64, 128, 256, 512, 1024\}$ | $B = 2,\ S = 256$ |
| Qwen-3.5 1B Decode | `torch_npu` + Transformers | one-token query, $D = 640$, $N_H = 8$, $D_H = 80$, paged KV cache | $B = 2$, $L_{\text{KV}} \in \{32, 64, 128, 256, 512, 1024\}$ | $B = 16,\ L_{\text{KV}} = 256$ |

HSTU generative recommendation [38], LLaMA-3 1B prefill and decode, and Qwen-3.5 1B prefill and decode. Table 4 provides the fixed model configurations, dynamic-shape range, and representative shapes.

For the **GPU platform**, we target FlashInfer [37], a state-of-the-art attention kernel library used in vLLM, and evaluate binary size reduction for Llama and Qwen model families.

*Implementation Details.* All evaluations for both operators and models are conducted using PyTorch 2.6.0. Dynamic dimensions are explicitly marked using the `Dynamic` API, and we utilize PyTorch Dynamo with dynamic compilation enabled. For the operator benchmark, we compare **TilingInfer** against the following baselines: **Baseline**, standard offline compilation with LLVM passes without kernel specialzation; **LLVM-spec**, which performs constant propagation within standard LLVM, but without early-exit identification and TilingInfer's inter-procedural constant propagation. The derived schedule parameters are then used to specialize kernels same as TilingInfer; **Optimal**, serves as an oracle reference. **Optimal** is computed by specializing the kernel with all constant schedule parameters derived for each individual shape configuration. It is unsuitable for production inference because dynamic inputs require per-shape recompilation, incurring prohibitive compilation overhead.

## 4.2 Performance Results on Ascend

*4.2.1 Operator Performance Analysis.* Figure 10 presents the normalized speedup of the evaluated operators across three methods: **LLVM-spec**, **TilingInfer**, and **Optimal**. All results are normalized to the **Baseline** (standard offline compilation).

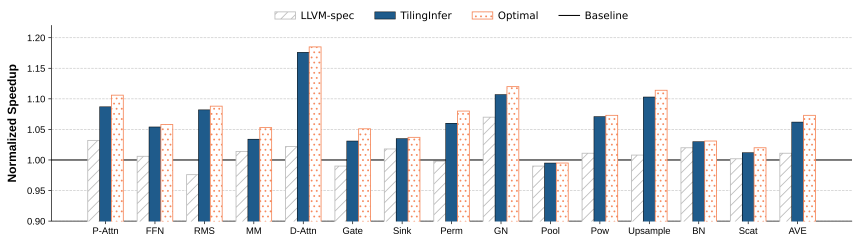



Fig. 10. Overall normalized performance of **TilingInfer** compared to compiler baselines

Overall, **TilingInfer** achieves a geometric mean speedup of **1.06**× across all operators, substantially outperforming the **LLVM-spec** baseline (1.01×). Crucially, the speedup ratio of **TilingInfer** is close to that of **Optimal** (within 0.5% on average). This demonstrates that **TilingInfer** successfully specializes the majority of schedule parameters, leaving only a few residual parameters that have negligible impact on performance. These results validate that **TilingInfer** achieves effective automatic kernel specialization for NPU operators.

We observe distinct performance behaviors across operator categories: **Attention mechanisms** (P-Attn, D-Attn) show the most significant gains, **1.08**× for P-Attn and **1.17**× for D-Attn, because these schedule-generic kernels have a large number of schedule parameters. RMS and GN also benefit notably (**1.08**× and **1.11**×), as constant specialization eliminates redundant boundary checks and simplifies division operations.

*4.2.2 Representative End-to-End Inference Scenarios.* The shape configurations of the six representative inference scenarios are summarized in Table 4, and their operator calls are classified in Table 5. We divide the calls into seven categories. MatMul/GMM, Attention, and Norm invoke CANN kernels from a schedule-generic implementation and together account for approximately 40%–94% of model-inference latency. The other four categories use kernels generated by TVM [6], an AI compiler that provides a Python-embedded DSL for describing operator implementations. These kernels have simple schedules, with schedule parameters already specialized during code generation, so TilingInfer provides no further speedup for them.

Figure 11 shows that TilingInfer's end-to-end speedup generally decreases as the runtime shape grows because tensor computation occupies an increasing fraction of execution time. DeepFM and the two decode scenarios obtain the larger gains, averaging approximately 1.10×, whereas HSTU and the two prefill scenarios average approximately 1.03×. Relative to Optimal's per-shape full specialization, TilingInfer achieves comparable geometric-mean speedups—1.05×

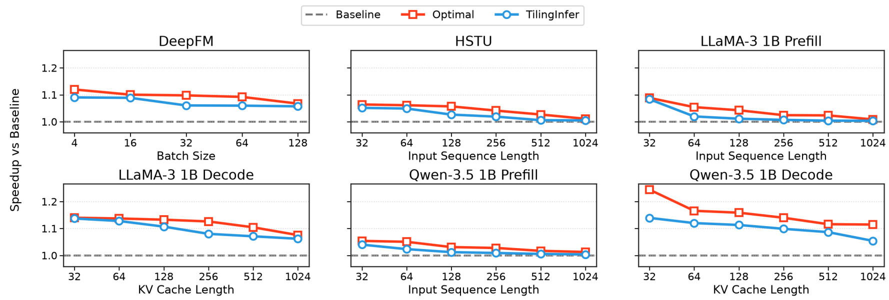



Fig. 11. End-to-end speedup versus runtime shape for six representative ML inference scenarios.

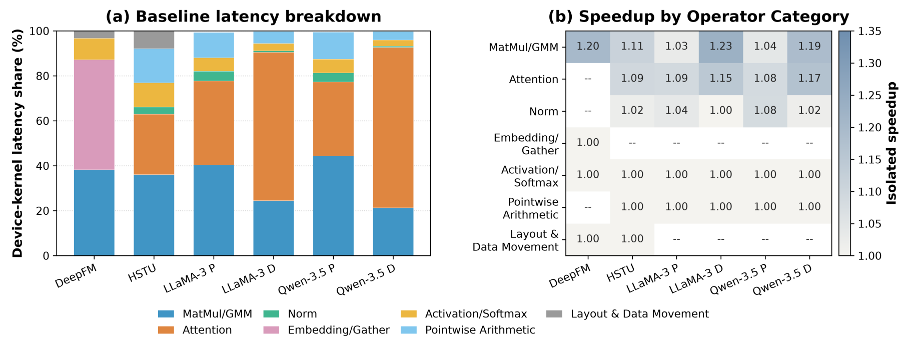



Fig. 12. Device-kernel latency analysis for six inference scenarios at representative shapes.

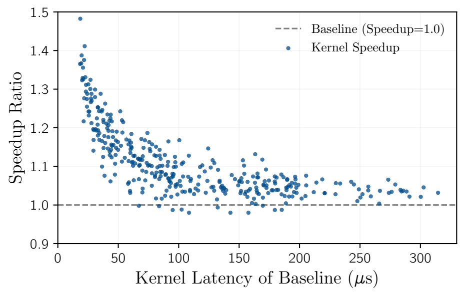
versus 1.08× across all six scenarios and 1.10× versus 1.14× across the two decode scenarios; although faster, Optimal recompiles the entire operator for every shape, taking tens of minutes each and thus being impractical in production.

Table 5. Operator categories, implementation sources, and TilingInfer outcomes for the six representative models.

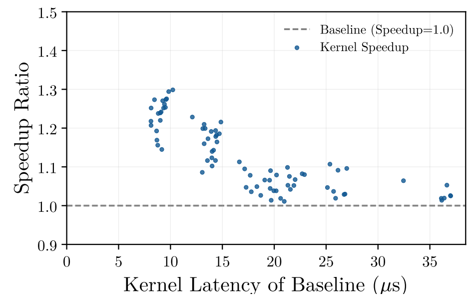
| Category | Operators | Implementation source / outcome |
|---|---|---|
| MatMul/GMM | Matrix Multiplication (MM), Batched Matrix Multiplication (BMM), Grouped Matrix Multiplication (GMM) | CANN operator library; TilingInfer specialization benefits |
| Attention | P-Attn, D-Attn | |
| Norm | RMSNorm, LayerNorm, GroupNorm | |
| Embedding/Gather | Embedding, Gather | AI-compiler-generated kernels (TVM Python DSL); no TilingInfer specialization benefit |
| Activation/Softmax | ReLU, SiLU/Swish, GELU, Sigmoid, Tanh, Softmax, Reduce/Mean | |
| Pointwise Arithmetic | Add, Sub, Mul, Div, Pow, Rsqrt, Maximum/Minimum, Zero | |
| Layout & Data Movement | Concat, Slice, Transpose, Cast, Copy/TensorMove, Reshape/AsStrided | |

Figure 12 attributes the model-level gains primarily to Attention and MatMul/GMM. In particular, the MatMul calls in DeepFM and decode phase of LLMs have relatively small computation sizes, so specialization removes a larger fraction of their tiling and control overhead and produces higher operator speedups. Attention supplies the other major source of improvement, especially in the decode phase. By contrast, the four AI-compiler-generated categories remain at 1.00×, confirming that TilingInfer's benefits are limited to expert-written operators.

*4.2.3 Comparison between **TilingInfer** and specialization based on Standard LLVM.* Table 6 shows the trade-off between compilation overhead and optimization effects. **LLVM-spec** incurs minimal overhead (less than 0.60s) due to its imprecise analysis. In contrast, **TilingInfer** exhibits a moderate increase (0.24s to 2.70s) due to field-sensitive analysis. Given that this compilation is a one-off cost during deployment, we consider this overhead acceptable, as it enables the backend compiler to generate considerably more efficient binaries (1.06× speedup). Moreover, **TilingInfer** avoids employing path-sensitive analysis through early-exit path identification, keeping compilation overhead acceptable.




(a) D-Attn




(b) MM

Fig. 13. Kernel speedup versus Baseline device-kernel latency for D-Attn and MM. Blue points report the speedup of TilingInfer relative to Baseline.

Table 6. Detailed comparison of compilation overhead and parameter specialization. **Total Params** indicates the number of schedule parameters defined in the operator program. **Time** is total compilation time in seconds. **Diff** is the time increment relative to Baseline. **Derived #** is the count of invariant schedule parameters inferred by the compiler.

| Operator | | P-Attn | D-Attn | MM | FFN | RMS | Gate | Sink | Perm | GN | Pool | Pow | Upsample | BN | Scat |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Total Params** | | **264** | **389** | **72** | **123** | **15** | **60** | **20** | **42** | **23** | **17** | **10** | **143** | **22** | **20** |
| **Baseline** | Time (s) | 24.50 | 19.80 | 42.10 | 4.52 | 3.12 | 1.85 | 5.10 | 1.24 | 6.15 | 1.35 | 0.82 | 2.95 | 3.30 | 1.15 |
| **LLVM-Spec** | Time (s) | 25.10 | 20.15 | 42.45 | 4.65 | 3.20 | 1.92 | 5.30 | 1.30 | 6.40 | 1.40 | 0.88 | 3.10 | 3.35 | 1.20 |
| | *Diff* | (+0.60) | (+0.35) | (+0.35) | (+0.13) | (+0.08) | (+0.07) | (+0.20) | (+0.06) | (+0.25) | (+0.05) | (+0.06) | (+0.15) | (+0.05) | (+0.05) |
| | Derived # | 4 | 3 | 3 | 2 | 1 | 2 | 0 | 0 | 1 | 0 | 0 | 3 | 2 | 1 |
| **TilingInfer** | Time (s) | 27.20 | 21.05 | 42.95 | 5.42 | 3.58 | 2.25 | 6.95 | 1.65 | 7.25 | 1.62 | 1.15 | 3.55 | 3.54 | 1.52 |
| | *Diff* | (+2.70) | (+1.25) | (+0.85) | (+0.90) | (+0.46) | (+0.40) | (+1.85) | (+0.41) | (+1.10) | (+0.27) | (+0.33) | (+0.60) | (+0.24) | (+0.37) |
| | Derived # | 242 | 352 | 60 | 110 | 11 | 30 | 13 | 28 | 17 | 15 | 5 | 82 | 15 | 14 |

## 4.3 Device-Level Attribution

Table 7. Performance Comparison: Speedup and detailed hardware metrics across **Baseline** (B), **LLVM-spec** (L), and **TilingInfer** (T). Best values are bolded. Reflecting our goal to maximize time spent on tensor computation units (Cube and Vector), ↓ denotes lower is better for metrics irrelative to tensor computations. ↑ denotes higher is better for speedup ratio.

| Op | Speedup (↑) | | | I-Cache Miss (%) (↓) | | | Scalar Ratio (%) (↓) | | | Mem Read Ratio (%) (↓) | | | Mem Write Ratio (%) (↓) | | | Binary Size(KB) (↓) | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | **B** | **L** | **T** | **B** | **L** | **T** | **B** | **L** | **T** | **B** | **L** | **T** | **B** | **L** | **T** | **B** | **L** | **T** |
| P-Attn | 1.00 | 1.03 | **1.08** | 16.40 | 16.30 | **11.39** | 91.23 | **90.29** | 90.33 | **8.26** | 8.30 | 8.95 | 3.94 | **3.81** | 4.11 | 363.81 | 360.76 | **310.12** |
| FFN | 1.00 | 1.01 | **1.05** | 3.89 | 4.12 | **0.00** | 84.72 | 84.52 | **83.60** | 6.42 | 7.24 | **5.49** | **0.10** | 0.11 | 0.11 | 68.79 | 68.89 | **54.91** |
| RMS | 1.00 | 0.98 | **1.08** | **3.00** | 3.01 | 3.36 | 97.89 | 97.82 | **88.57** | 6.83 | 7.27 | **6.72** | **2.90** | 3.02 | 3.13 | 36.32 | 36.32 | **33.23** |
| MM | 1.00 | 1.01 | **1.03** | **3.12** | 3.42 | 3.41 | 84.70 | 84.74 | **84.68** | **42.66** | 47.10 | 45.95 | 0.00 | 0.00 | 0.00 | 101.38 | 101.26 | **100.84** |
| D-Attn | 1.00 | 1.02 | **1.17** | 4.45 | 5.51 | **0.00** | 77.98 | 77.08 | **74.43** | 6.42 | 6.44 | **4.91** | 2.46 | **1.60** | 1.79 | 58.30 | 58.59 | **45.05** |
| Gate | 1.00 | 0.99 | **1.03** | 0.00 | 0.00 | 0.00 | **96.98** | **96.98** | 96.99 | 25.00 | 24.82 | **16.43** | 1.17 | **1.05** | 1.36 | 17.10 | **17.09** | **17.09** |
| Sink | 1.00 | 1.02 | **1.03** | 0.00 | 0.00 | 0.00 | 52.46 | 52.96 | **50.34** | 4.41 | 4.68 | **4.34** | 18.60 | 18.14 | **17.88** | 47.54 | 47.56 | **47.11** |
| Perm | 1.00 | 1.00 | **1.08** | 0.00 | 0.00 | 0.00 | **92.34** | 92.58 | 92.62 | 10.55 | **9.31** | 10.52 | 5.84 | 6.25 | **5.69** | 24.02 | 24.14 | **14.40** |
| GN | 1.00 | 1.07 | **1.11** | 0.80 | 1.20 | **0.00** | 99.10 | 98.94 | **98.90** | 24.98 | **18.46** | 18.83 | 14.20 | **9.29** | 9.31 | 220.76 | 220.56 | **193.80** |
| Pool | 1.00 | 0.99 | 0.99 | 0.00 | 0.00 | 0.00 | 97.27 | 97.21 | **97.14** | 45.01 | 45.01 | 45.01 | 0.00 | 0.00 | 0.00 | 13.35 | 13.35 | **13.24** |
| Pow | 1.00 | 1.01 | **1.07** | 0.00 | 0.00 | 0.00 | 59.45 | 57.60 | **53.11** | 44.64 | 45.48 | 40.16 | 2.72 | 2.80 | 3.00 | 24.58 | 24.58 | 24.31 |
| Upsample | 1.00 | 1.00 | **1.10** | 9.83 | 9.98 | **0.00** | 99.50 | 99.51 | **98.42** | 16.45 | 17.38 | **13.87** | **1.76** | 2.08 | 2.15 | 56.84 | 56.84 | **37.54** |
| BN | 1.00 | 1.01 | **1.03** | 0.00 | 0.00 | 0.00 | **96.60** | 96.69 | 96.82 | 64.86 | 66.16 | **60.02** | 22.94 | **22.13** | 22.44 | 15.73 | 15.69 | **15.61** |
| Scat | 1.00 | 1.00 | **1.01** | 24.38 | 23.42 | **22.61** | **97.39** | 98.06 | 97.93 | 5.44 | 4.36 | **1.41** | **0.90** | 1.29 | 1.12 | 24.11 | 23.99 | **23.40** |

Table 7 and Figure 13 show that TilingInfer does not produce uniform device-level gains; benefiting kernels fall into two mechanism-driven classes, whereas a third class is largely insensitive. For Upsample, D-Attn, FFN, and P-Attn, gains mainly arise because invariant propagation exposes constant branches and loop structure, enabling dead-code elimination and control-flow simplification; the resulting smaller instruction footprint reduces instruction-fetch and I-Cache pressure. A second class, including RMS, Pow, Gate, and GN, benefits primarily from simplified runtime calculation and memory scheduling. Resolving loop bounds, offsets, and schedule parameters removes scalar loads and address or control computations, exposes immediate operands, and enables access merging, loop unrolling, and more effective instruction scheduling. In contrast, Pool and Scat are largely insensitive: Pool is already compact with little instruction-fetch overhead, whereas Scat is dominated by memory work, so instruction-stream simplification does not shorten the critical path. Across all three classes, specialization mainly removes a roughly fixed amount of control and scheduling overhead. Its relative effect is therefore strongest for short-running kernels and diminishes as tensor computation or memory traffic dominates. The key determinant is not whether a parameter is static, but whether resolving it eliminates work on the kernel's performance-critical path.

## 4.4 Sensitivity to Resolved Schedule Parameters

**(a) D-Attn** $\text{Query} \times \text{Key}^{\mathsf{T}}$

$$\text{Output}_{b,h} = \text{Query}_{b,h}\text{Key}^{\mathsf{T}}_{b,h}$$

$\text{Query}_{b,h} : [1, D_{head}]$: Query row of attention head $h$ for request $b$ in the batch.

$\text{Key}_{b,h} : [T_b, D_{head}]$: cached Key rows of attention head $h$ for request $b$ in the batch.

$\mathbf{T} = [T_0, \ldots, T_{B-1}]$: per-request KV lengths (runtime input).

**(b) Representative schedule parameters for** $\text{Query} \times \text{Key}^{\mathsf{T}}$ (34 total)

**Control logic (6 total)**

$T_N$: Key rows in one output tile.
$P$: Key rows in one cache page.
$T_D$: Key columns in one narrow load.

**Per-core tile dimensions/addressing (14 total)**

$M_c$: Output-row extent assigned to one core.
$K_c$: Reduction-column extent assigned to one core.

**Core-loop scheduling/memory resources (14 total)**

$N_c$: Number of active compute cores.
$n_K$: Number of Key buffers.

**(c) Tiled BMM pseudocode for** $\text{Query} \times \text{Key}^{\mathsf{T}}$

```
function BMM(Query, Key, T; T_N, P, T_D, M_c, K_c, N_c, n_K)
  configure N_c cores and n_K Key buffers
  parallel for c = 0, ..., N_c − 1
    for tile τ with T_N Key rows × K_c columns assigned to core c
      // n_0: first Key row; k_0: first reduction column; T_b: request b's KV length
      (b, h, n_0, k_0) ← TileOffset(τ; M_c, T_N, K_c, T)
      n ← min(T_N, T_b − n_0)
      for (p, r, u) in PageFragments(Key, b, n_0, n; P)
        if T_N = P
          one-copy load Key[p, r : r + u, h, k_0 : k_0 + K_c] → KeyBuf[next(n_K)]
        else (T_N ≠ P)
          for d_0 = k_0, k_0 + T_D, ..., k_0 + K_c
            append load Key[p, r : r + u, h, d_0 : d_0 + T_D] → KeyBuf[next(n_K)]
      load Query[b, 0, h, k_0 : k_0 + K_c] → QueryBuf
      Output_tile += MMAD(QueryBuf, KeyBuf)
```

**TileOffset**: maps tile $\tau$ to start coordinate $(b, h, n_0, k_0)$: request, head, Key row, and reduction column.
**PageFragments**: resolves $n$ logical Key rows through the page table as non-crossing fragments $(p, r, u)$.
**MMAD**: multiplies QueryBuf by KeyBuf and accumulates the partial result into $\text{Output}_{\text{tile}}$.

Fig. 14. Tiled D-Attn BMM. Panel (b) lists seven representative schedule parameters; the three groups contain 6, 14, and 14 schedule parameters in total. Matching backgrounds in panel (c) mark their use sites and correspond to the groups evaluated in Figure 15.

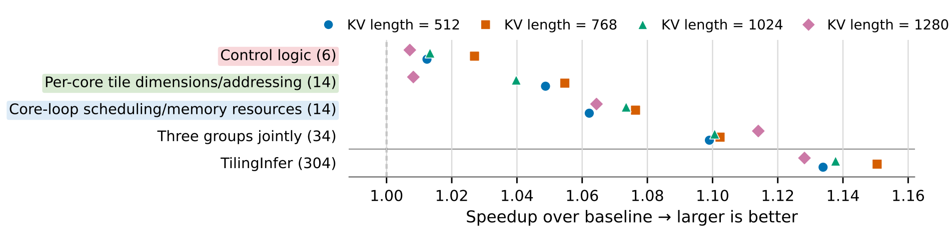



Fig. 15. Sensitivity to resolved schedule parameters across runtime shapes. Numbers in parentheses give the number of specialized schedule parameters. Each point shows the speedup at one KV length; the first three label backgrounds match the groups in Figure 14.

This section analyzes whether specializing different groups of schedule parameters yields different benefits: can one group provide a substantially larger gain than the others?

For D-Attn, $\text{Query}_{b,h}\text{Key}^{\mathsf{T}}_{b,h}$ multiplies the Query row of request $b$ and attention head $h$ by its cached Key rows, while $\mathbf{T}$ stores each request's KV length. Its 34 schedule parameters form Control logic (6), Per-core tile dimensions/addressing (14), and Core-loop scheduling/memory resources (14). Control logic covers the branches and loop bounds that select how Key data are loaded. Per-core tile dimensions/addressing specifies how much work each core owns and maps tile indices to tensor addresses. Core-loop scheduling/memory resources controls how tiles are assigned to active cores and how on-chip buffers are allocated and reused inside the core loop. Figure 14(b) lists seven representative examples. In panel (c), $T_N$ and $K_c$ split Key rows and reduction columns into tiles, while $M_c$ and $N_c$ distribute the tiles to cores. TileOffset uses these extents and $\mathbf{T}$ to recover $(b, h, n_0, k_0)$, the tile's starting coordinate. The schedule parameters $T_N$, $P$, and $T_D$ select one page-aligned load or several narrow page-table loads, while $n_K$ selects the Key buffer. BMM then loads the matching Query slice and computes the tile.

Overall, Figure 15 shows that specialization value depends more on a parameter's use site than on parameter count. Across KV lengths 512–1280, the 14 Core-loop scheduling/memory-resource parameters achieve the largest isolated geometric-mean speedup, 1.07×, by configuring per-core work and Key-buffer resources on the inner BMM path. The 14 Per-core tile dimension/addressing parameters reach 1.04×, while the 6 Control-logic parameters reach 1.02×. Thus, count alone poorly predicts specialization value.

The three isolated gains sum to 12.2%, whereas jointly specializing all 34 parameters yields 1.10× (10.4%), or 85.0% of that sum, indicating interaction and diminishing returns. TilingInfer specializes 304 stable parameters and reaches 1.14×. Although the 34 Query × $\text{Key}^{\mathsf{T}}$ parameters comprise only 11.0% of these fields, they recover 75.0% of the full gain; the remaining 270 add only 3.4 percentage points. Because these 34 parameters control D-Attn's core BMM in Figure 14(c), specializing its hot-path address and control flow is more valuable than resolving constants on colder paths.

## 4.5 Binary-Size Trade-off

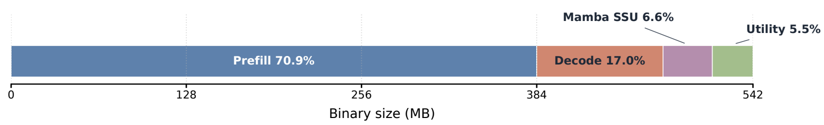



Fig. 16. Full AOT archive breakdown.

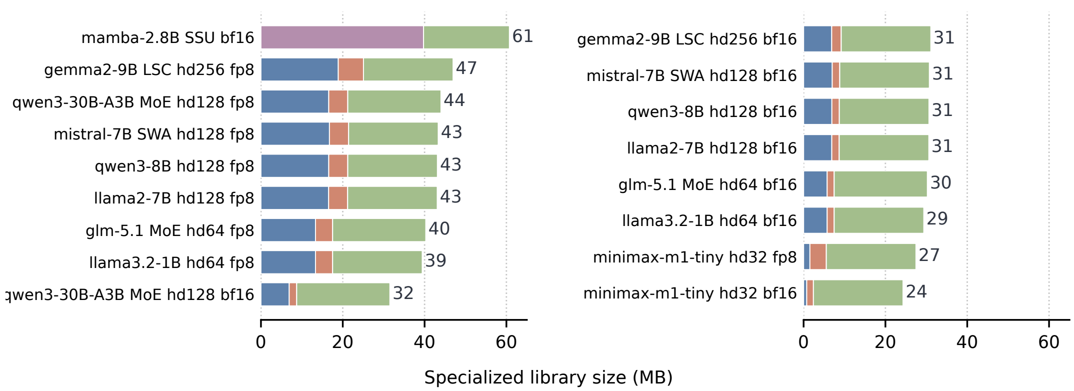



Fig. 17. Per-model FlashInfer footprint.

We apply path-liveness information to FlashInfer and retain the shared objects reachable by each of 17 model variants, including standard, FP8 KV-cache, MoE, sliding-window, and logits-soft-cap attention as well as Mamba Selective-State-Update (SSU) kernels. The universal AOT archive contains 247 shared objects and occupies 542 MB. As shown in Figure 16, Prefill and Decode kernels account for 87.9% of its footprint, with Prefill alone contributing 70.9%. Prefill kernels dominate because they have high computational cost, complex implementations, and numerous schedule-parameter combinations that are enumerated and pre-specialized in AOT shared libraries.

As shown in Figure 17, the retained per-model libraries occupy 24.3–60.7 MB, reducing the universal archive by 88.8–95.5%. To notice, the experiment is conducted without cross-service deduplication and the extrema place the storage break-even point at roughly 8–22 model services. In the worst case, serving 8 models like mamba would require 8×60.7 MB = 485.6 MB, which is still smaller than the universal archive of 542 MB. A universal archive therefore favors diverse multi-model deployments, whereas per-model pruning favors serving modest model sets.

TilingInfer removes most of the binary footprint from Prefill and Decode kernels because these kernels enumerate and pre-specialize many combinations of template schedule parameters. After pruning these kernels, the retained libraries consist mainly of various complex Utility kernels and SSU kernels.

# 5 Related Work

## 5.1 Partial Evaluation and Interprocedural Specialization

Constant propagation, partial evaluation, and application specialization are established compiler techniques. Sparse conditional constant propagation combines value and executable-edge reasoning [35], while systems such as TRIMMER specialize and debloat programs from deployment information [25]. C++ templates can also be understood as a manual form of partial evaluation [34]. TilingInfer builds on these foundations rather than claiming a new constant-propagation primitive. Its contribution is to recover a typed C++ entry state that is present in a framework Graph IR but absent from the operator library's LLVM IR, propagate that state through pointer-rich host code, and preserve precision across shape-checker-induced early-exit paths.

## 5.2 AI Compilers and Kernel Generation

Dynamic-shape AI compilers such as DietCode [39], Nimble [27], BladeDISC [42], Relax [17], and CoRa [11] transform compiler-visible tensor programs and decide how to lower dynamic dimensions. Kernel languages and scheduling systems such as Triton [31], TileLang [30], and TVM/AutoTVM [6] help operator developers generate kernels and search or autotune schedules for the target shapes. Once a schedule is selected, its schedule parameters are encoded directly in the generated implementation.

LLM-assisted systems change how the target program is produced but retain this generation or translation paradigm. QiMeng-Xpiler combines LLM-guided transformations, symbolic repair, and hierarchical autotuning to translate tensor programs across accelerator programming interfaces [10]. KernelBench evaluates whether LLMs can generate correct and efficient GPU kernels from ML workloads [22]. These systems generate or translate an implementation and then optimize its transformation or schedule choices. They do not reconstruct deployment constants inside an unchanged expert-written operator library to partially evaluate scalar LLVM instructions or prune unreachable precompiled variants.

TilingInfer therefore has a different optimization object. It preserves the expert-written kernel, including its vendor-specific control flow, data movement, and intrinsic calls, and specializes standard scalar LLVM instructions such as loads, stores, arithmetic, branches, and address calculations. This distinction is methodological: comparing a generated kernel with an expert-written kernel would mix schedule and implementation quality with the effect of specialization that TilingInfer is designed to isolate.

## 6 Discussion

**Workload-dependent benefits.** Our experiments show that the magnitude of TilingInfer's benefit depends strongly on workload characteristics. Recommendation and decode workloads benefit most because Attention and MatMul expose profitable invariant schedules, yielding about 10% average inference speedup. Schedule-fixed expert-written libraries also benefit from removing unreachable template instances. This per-model pruning is most attractive when a deployment serves fewer models, because each specialized binary can discard a larger fraction of the universal library. By contrast, HSTU and prefill remain compute-heavy, so specializing their core Attention and MatMul kernels removes little of the total work and improves end-to-end latency by only about 2%. Some kernels in the current open-source baselines also cannot be further specialized by TilingInfer, further limiting end-to-end gains.

**Platform applicability.** Our schedule-generic kernel-specialization implementation currently targets Ascend. In the open-source libraries we surveyed for other accelerators, expert-written kernels predominantly follow the schedule-fixed design, leaving no comparable schedule-generic baseline to evaluate. Ascend exposes many programmable hardware levels; achieving high performance therefore relies on numerous runtime schedule parameters, which motivates schedule-generic libraries and makes automatic specialization particularly relevant.

**Optimization scope and limitations.** TilingInfer applies to existing expert-written operator libraries: it specializes schedule-generic kernels or removes unreachable instances from schedule-fixed libraries. This differs from AI compilers, which generate implementations and search schedules; their code-generation pipelines do not optimize an existing C++ expert library in place.

Besides, TilingInfer analyzes host code only to recover constants and launch-site liveness, then optimizes reachable device kernels; it does not rewrite host code. Host control flow is complex, CPUs already execute it efficiently, and profitable host specialization would require a more selective policy than device-kernel constant propagation. Production NPU Graph execution further suppresses CPU overhead. In layered Transformers, one host path is typically executed once per decode step while its device kernels repeat across many layers, making kernel optimization the higher-value target.

## 7 Conclusion

We presented TilingInfer, which propagates model-specific Graph-IR constants through existing C++ operator libraries to specialize schedule-generic device kernels or prune unreachable schedule-fixed instances.

Across 14 Ascend operators, TilingInfer delivers a **1.06×** geometric-mean device-kernel speedup, with a maximum of **1.17×**; its largest end-to-end gains occur in recommendation and decode workloads whose Attention and MatMul schedules expose profitable invariants, averaging about 10%.

For the schedule-fixed FlashInfer library, per-model pruning reduces the 542 MB universal binary to 24.3–60.7 MB across 17 model configurations, an 88.8–95.5% reduction, demonstrating that graph-derived deployment knowledge can improve either device-kernel execution or binary footprint without synthesizing new schedules.

## References

[1] Martín Abadi, Paul Barham, Jianmin Chen, et al. 2016. TensorFlow: A system for large-scale machine learning. In 12th OSDI. USENIX Association, USA, 265–283.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4895–4901. doi:10.18653/v1/2023.emnlp-main.298

[3] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, David Berard, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Laurent Kirsch, Michael Lazos, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366

[4] Ascend. 2024. cann-ops-adv: An Advanced Fusion Operator Library for Ascend Hardware. https://gitee.com/ascend/cann-ops-adv A library of fusion operators based on Ascend hardware..

[5] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL] https://arxiv.org/abs/2004.05150

[6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (Carlsbad, CA, USA) (OSDI'18). USENIX Association, USA, 579–594.

[7] NVIDIA Corporation. 2017. CUTLASS: CUDA Template Library for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass. Accessed: 2024-07-09.

[8] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, Vienna, Austria, 14 pages. https://openreview.net/forum?id=mZn2Xyh9Ec

[9] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[10] Shouyang Dong, Jun Bi, Di Huang, Jiaming Guo, Jianxing Xu, Ruibai Xu, Xinkai Song, Yifan Hao, Ling Li, Xuehai Zhou, Tianshi Chen, Qi Guo, and Yunji Chen. 2025. QiMeng-Xpiler: Transcompiling Tensor Programs for Deep Learning Systems with a Neural-Symbolic Approach. In 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25). USENIX Association, Boston, MA, USA, 239–255. https://www.usenix.org/conference/osdi25/presentation/dong

[11] Pratik Fegade, Tianqi Chen, Phillip Gibbons, and Todd Mowry. 2022. The CoRa tensor compiler: Compilation for ragged tensors with minimal padding. Proceedings of Machine Learning and Systems 4 (2022), 721–747.

[12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence. International Joint Conferences on Artificial Intelligence, Melbourne, Australia, 1725–1731. doi:10.24963/ijcai.2017/239

[13] Shengyu Liu Jiashi Li. 2025. FlashMLA: Efficient Multi-head Latent Attention Kernels. https://github.com/deepseek-ai/FlashMLA.

[14] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In Proceedings of the 50th annual international symposium on computer architecture. Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/3579371.3589350

[15] Shuangxiang Kan, Yuhao Gao, Zexin Zhong, and Yulei Sui. 2024. Cross-Language Taint Analysis: Generating Caller-Sensitive Native Code Specification for Java. IEEE Trans. Softw. Eng. 50, 6 (June 2024), 1518–1533. doi:10.1109/TSE.2024.3392254

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[17] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery, New York, NY, USA, 998–1013. doi:10.1145/3676641.3716249

[18] Heng Liao, Jiajin Tu, Jing Xia, and Xiping Zhou. 2019. Davinci: A scalable architecture for neural network computing. In 2019 IEEE Hot Chips 31 Symposium (HCS). IEEE Computer Society, IEEE, Cupertino, CA, USA, 1–44.

[19] Raphaël Monat, Abdelraouf Ouadjaout, and Antoine Miné. 2021. A Multilanguage Static Analysis of Python Programs with Native C Extensions. In Static Analysis: 28th International Symposium, SAS 2021, Chicago, IL, USA, October 17–19, 2021, Proceedings (Chicago, IL, USA). Springer-Verlag, Berlin, Heidelberg, 323–345. doi:10.1007/978-3-030-88806-0_16

[20] NVIDIA. 2021. cuBLAS library (11.2). Online at https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf. https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf

[21] OpenXLA Project. 2024. StableHLO: High Level Operations for Machine Learning. https://openxla.org/stablehlo. Accessed: 2026-05-29.

[22] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] https://arxiv.org/abs/2502.10517

[23] Adam Paszke, Sam Gross, Francisco Massa, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In 32th NeurIPS. Curran Associates, Inc., Conference Location Unknown, 8024–8035.

[24] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: A New IR for Machine Learning Frameworks. In 2nd MAPL (Philadelphia, PA, USA). Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3211346.3211348

[25] Hashim Sharif, Muhammad Abubakar, Ashish Gehani, and Fareed Zaffar. 2018. TRIMMER: Application Specialization for Code Debloating. In Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. ACM, Montpellier France, 329–339. doi:10.1145/3238147.3238160

[26] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150 [cs.NE] https://arxiv.org/abs/1911.02150

[27] Haichen Shen, Jared Roesch, Zhi Chen, et al. 2021. Nimble: Efficiently compiling dynamic neural networks for model inference. In 4th MLSys, Vol. 3. Publisher Unknown, Conference Location Unknown, 208–222.

[28] Tensorflow. 2021. XLA: Optimizing Compiler for Machine Learning. https://www.tensorflow.org/xla.

[29] Chen Tianqi, Eddie Yan, et al. 2018. Learning to Optimize Tensor Programs. In 32nd NeurIPS. Publisher Unknown, Conference Location Unknown, 3393–3404.

[30] TileLang Team. 2024. TileLang: A Flexible and High-Performance DSL for Kernel Development. https://github.com/tile-ai/tilelang. Accessed: 2024.

[31] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In 3rd MAPL (Phoenix, AZ, USA) (MAPL 2019). Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[32] Hugo Touvron and Louis Martin. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems 30, "" (2017), "". No formal volume, number, or pages available.

[34] Todd L. Veldhuizen. 1998. C++ Templates as Partial Evaluation. arXiv:cs/9810010 [cs.PL] https://arxiv.org/abs/cs/9810010

[35] Mark N. Wegman and F. Kenneth Zadeck. 1991. Constant Propagation with Conditional Branches. ACM Trans. Program. Lang. Syst. 13, 2 (April 1991), 181–210. doi:10.1145/103135.103136

[36] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. arXiv:2407.10671 [cs.CL] https://arxiv.org/abs/2407.10671

[37] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In Proceedings of Machine Learning and Systems, Vol. 7. MLSys, Santa Clara, CA, USA, 17 pages. https://proceedings.mlsys.org/paper_files/paper/2025/file/dbf02b21d77409a2db30e56866a8ab3a-Paper-Conference.pdf

[38] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235). PMLR, Vienna, Austria, 58484–58509. https://proceedings.mlr.press/v235/zhai24a.html

[39] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Joshua Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. 2022. DietCode: Automatic optimization for dynamic tensor programs. Proceedings of Machine Learning and Systems 4 (2022), 848–863.

[40] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, Conference Location Unknown, 863–879. https://www.usenix.org/conference/osdi20/presentation/zheng

[41] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. Advances in neural information processing systems 37 (2024), 62557–62583.

[42] Zhen Zheng, Zaifeng Pan, Dalin Wang, Kai Zhu, Wenyi Zhao, Tianyou Guo, Xiafei Qiu, Minmin Sun, Junjie Bai, Feng Zhang, Xiaoyong Du, Jidong Zhai, and Wei Lin. 2023. BladeDISC: Optimizing Dynamic Shape Machine Learning Workloads via Compiler Approach. Proc. ACM Manag. Data 1, 3 (nov 2023), 1–29. doi:10.1145/3617327